

How Do LLM-Based Agents Represent Code Repositories? A Systematic Literature Review

ANONYMOUS AUTHOR(S)

Large Language Model (LLM)-based agents are rapidly transforming software engineering practice, yet the fundamental question of how these systems represent entire code repositories remains poorly understood. A repository is not merely a collection of files; it is a rich, interconnected artifact encompassing call graphs, dependency networks, semantic relationships, historical evolution, and domain-specific conventions. Without effective repository representation, LLM-based agents cannot reliably perform complex tasks such as cross-file code completion, project-level bug localization, feature implementation, code translation, or automated issue resolution.

This paper presents the first systematic literature review (SLR) focused specifically on *repository representation paradigms* (PARs) in LLM-based software engineering. Following the Kitchenham and Charters guidelines for SLR in software engineering and the PRISMA 2020 reporting standard, we analyzed 160 primary studies published between 2020 and 2026. From this corpus, we derive a seven-pillar taxonomy of repository representation paradigms: (PAR-1) dense token embeddings, (PAR-2) retrieval-augmented generation, (PAR-3) graph-based structural representations, (PAR-4) dynamic and execution-based context, (PAR-5) summarization and compression, (PAR-6) memory-augmented agent architectures, and (PAR-7) multi-step agentic frameworks.

We map each paradigm bidirectionally against seven software engineering tasks and evaluate their trade-offs across five dimensions: context coverage, retrieval latency, training cost, scalability, and benchmark availability. Our findings reveal that while PAR-2 (RAG) dominates current practice, PAR-3 (graphs) and PAR-7 (agentic) are rapidly gaining adoption for complex tasks. We identify critical gaps including the absence of longitudinal benchmarks, under-explored multi-paradigm hybrids, and the need for energy-aware evaluation metrics. This survey provides researchers and practitioners with a comprehensive reference for selecting and designing repository representation strategies for LLM-based software engineering agents.

CCS Concepts: • **Software and its engineering** → **General programming languages**; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: systematic literature review, large language models, code repositories, repository representation, retrieval-augmented generation, software engineering agents, code completion, bug localization

ACM Reference Format:

Anonymous Author(s). 2025. How Do LLM-Based Agents Represent Code Repositories? A Systematic Literature Review. *ACM Trans. Softw. Eng. Methodol.* 37, 4, Article 111 (August 2025), 42 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

The emergence of Large Language Models (LLMs) as core components of software engineering toolchains represents one of the most significant shifts in how software is designed,

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/8-ART111

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

written, maintained, and evolved. Systems such as GitHub Copilot, Amazon CodeWhisperer, Cursor, and a growing ecosystem of research prototypes now assist developers with tasks ranging from single-line autocomplete to full-feature implementation, test generation, bug localization, and automated issue resolution. Yet beneath these impressive capabilities lies a deceptively hard sub-problem: *how should an LLM-based agent represent an entire software repository?*

A modern software repository is far more than a flat collection of source files. It encompasses tens of thousands of functions distributed across hundreds of modules, intricate dependency hierarchies, documented interfaces, test suites, configuration files, commit histories, issue trackers, and implicit architectural conventions accumulated over years of collaborative development. The informational content of a non-trivial open-source project routinely exceeds ten million tokens—orders of magnitude beyond the context windows offered by even the most capable contemporary LLMs. Representing this richness faithfully, efficiently, and in a form that an LLM can exploit is the foundational challenge that all repository-aware AI systems must address.

1.1 Motivation and Problem Statement

The importance of repository representation becomes immediately apparent when one examines the failure modes of naive approaches. Early experiments with LLM-based code completion revealed that models trained solely on individual files or functions failed catastrophically on cross-file dependencies: they could not locate imported symbols, could not infer the contract of a called function defined elsewhere in the repository, and could not respect project-specific naming conventions that differ from open-source norms [25, 76]. The fundamental issue is one of context: the information needed to complete a code fragment correctly is often not in the immediate vicinity of the cursor, but distributed across the repository.

This observation has motivated a rapidly expanding body of research on repository-level software engineering with LLMs. Researchers have proposed techniques ranging from static retrieval of relevant snippets [81, 149] to rich graph-based representations of code structure [39, 77], from hierarchical summarization [23, 87] to multi-step agentic frameworks that dynamically navigate repository structure [57, 141, 158]. This body of work has grown to encompass hundreds of primary studies published in premier venues including ICSE, FSE, ASE, TOSEM, ACL, EMNLP, ICLR, NeurIPS, and ICML.

Despite this explosion of work, the field lacks a principled taxonomy of *repository representation paradigms*. Existing surveys of LLM-based software engineering [48, 53, 155] provide broad coverage of tasks and models but do not systematically examine the representation dimension. Reviews of code completion [51] focus on a single task and a single paradigm. Reviews of retrieval-augmented generation for code [118] treat retrieval as the *only* representation strategy, overlooking graph-based, embedding-based, agentic, and summarization-based alternatives. This fragmented picture makes it difficult for both researchers designing new systems and practitioners choosing deployment strategies to reason about the design space in a principled way.

1.2 The Repository Representation Problem

We define *repository representation* as the process by which an LLM-based agent constructs and maintains a structured or unstructured encoding of the information contained in a software repository, and makes that encoding available—directly or indirectly—to the LLM’s inference process. This definition is deliberately broad because the literature has produced

a diverse menagerie of strategies, each making different trade-offs among context coverage, computational cost, inference latency, scalability, and task-specific performance.

The core challenge can be decomposed into three sub-problems. First, there is the *indexing problem*: given a repository of N files containing T tokens in total, how does one construct a data structure that admits efficient queries of the form “given the current editing context c , find the k most relevant fragments”? The answer depends critically on what “relevant” means, which is itself task-dependent. For code completion, the most relevant fragments are those most likely to contain symbols, types, or patterns needed at the cursor. For bug localization, relevance is defined by causal proximity to the reported failure. For code translation, relevance includes type definitions, library bindings, and idiomatic patterns in the target language.

Second, there is the *selection problem*: given an index of relevant fragments, how does one select and rank a subset that fits within the LLM’s context window? The context window is simultaneously the most precious and most constrained resource in LLM-based software engineering. Overfilling it with irrelevant context degrades generation quality and increases inference cost; underfilling it wastes representational capacity and omits crucial information. The selection problem is compounded by the well-documented phenomenon of “lost in the middle”: LLMs disproportionately attend to tokens at the beginning and end of their context window, making the ordering of retrieved fragments non-trivially important [146].

Third, there is the *grounding problem*: once a representation has been selected and placed in the LLM’s context window, how does the model accurately interpret it? Different representations—raw source code, API documentation, call graph adjacency lists, compressed embeddings, natural language summaries—make different demands on the model’s ability to extract and apply the encoded information. A representation that is maximally information-dense from an information-theoretic perspective may be opaque to a language model trained primarily on natural language and syntactically structured source code.

1.3 Research Questions

This systematic literature review is organized around five research questions that together cover the full scope of the repository representation problem:

- RQ1** *What paradigms have been proposed for representing code repositories in LLM-based software engineering systems?* This question drives the construction of our seven-pillar taxonomy. We seek to enumerate and characterize the distinct representational strategies that appear in the literature, identify their key design choices, and trace their historical development from early dense embedding approaches through contemporary multi-agent frameworks.
- RQ2** *How do repository representation paradigms map onto software engineering tasks?* The appropriateness of a given paradigm is not universal; different tasks impose different requirements on the representation. We investigate which paradigms are most commonly applied to which tasks, and whether there are systematic associations between paradigm choice and task characteristics such as locality, complexity, and interaction with the execution environment.
- RQ3** *What are the empirical performance characteristics of each paradigm, and how do they compare across standardized benchmarks?* We synthesize quantitative results from primary studies to characterize the performance envelope of each paradigm, identify

which benchmarks are most widely used and thus best-suited for cross-study comparisons, and assess the degree to which benchmark results are reproducible and generalizable.

RQ4 *What are the key trade-offs among repository representation paradigms along dimensions of context coverage, retrieval latency, training cost, scalability, and benchmark availability?* We go beyond raw performance to examine the operational characteristics that determine the suitability of each paradigm for real-world deployment, highlighting scenarios where each paradigm excels and where it fails.

RQ5 *What are the open research challenges and future directions in repository representation for LLM-based software engineering?* We identify gaps in the current literature, propose concrete research agendas, and discuss emerging trends such as multi-paradigm hybridization, energy-aware evaluation, and adaptation to continuously evolving repositories.

1.4 Contributions

This paper makes the following primary contributions to the literature on LLM-based software engineering:

- (1) **Seven-Pillar PAR Taxonomy.** We present the first systematic taxonomy of repository representation paradigms (PAR-1–PAR-7), derived inductively from 160 primary studies using thematic synthesis [21]. Each paradigm is characterized by its representational substrate, indexing mechanism, query interface, context management strategy, and typical deployment pattern.
- (2) **Bidirectional Task–Paradigm Mapping.** We construct a comprehensive mapping from the seven SE tasks most commonly addressed in the literature to the PAR taxonomy, and conversely from each PAR to the tasks it most effectively supports. This mapping exposes systematic under-coverage (tasks with few paradigm options) and over-coverage (paradigms applied uniformly to all tasks without empirical justification).
- (3) **Multi-Dimensional Trade-Off Analysis.** We evaluate each PAR across five operational dimensions—context coverage, retrieval latency, training cost, scalability, and benchmark availability—providing a structured basis for paradigm selection.
- (4) **Research Agenda.** We distill our findings into a prioritized set of seven open research challenges, each supported by evidence from the reviewed literature, and propose concrete research directions for each challenge.
- (5) **Replication Package.** We provide a publicly available replication package containing the full search protocol, screening decisions, data extraction spreadsheet, and synthesis code.

1.5 Scope and Boundaries

This review focuses specifically on the *representation* of code repositories, not on LLM-based software engineering in general. We include studies that (a) target one or more repository-level software engineering tasks, (b) use an LLM or a pre-trained code model as the core inference engine, and (c) make explicit design choices about how repository content is represented to the model. We exclude studies that focus exclusively on single-function or single-file tasks with no cross-file context, studies that use LLMs purely as classifiers or reviewers without generating or modifying code, and studies that focus on code understanding for purposes other than software engineering (e.g., code plagiarism detection, malware analysis).

The temporal scope of this review is 2020–2026, capturing the period from the emergence of transformer-based code models (CodeBERT [31]) through the latest preprint literature available at the time of our search. This span encompasses both the “pre-LLM” era of specialized code representation models and the contemporary era of general large language models such as GPT-4, Claude, and Llama.

1.6 Paper Structure

The remainder of this paper is organized as follows. Section 2 describes the systematic review methodology, including the search protocol, inclusion/exclusion criteria, quality assessment, and data extraction procedures. Section 3 presents the seven-pillar PAR taxonomy, with detailed descriptions of each paradigm and representative works. Section 4 presents the bidirectional task–paradigm mapping and analyzes coverage patterns. Section 5 answers our five research questions with synthesized evidence from the primary studies. Section 6 compares this review with related surveys. Section 7 discusses threats to validity. Section 8 concludes with a summary of key findings and the prioritized research agenda. Appendices provide the full data extraction tables, quality assessment results, and the complete list of primary studies.

2 Methodology

This systematic literature review was conducted following the evidence-based software engineering guidelines of Kitchenham and Charters [58], the PRISMA 2020 reporting standard [98], and the ACM SIGSOFT Empirical Standards [4]. The review protocol was documented prior to data collection to reduce the risk of post-hoc selection bias. The full protocol and all intermediate artefacts are available in the replication package.

2.1 Research Question Refinement

The five research questions introduced in Section 1 were refined through an iterative process involving three rounds of piloting. In round one, two of the authors independently read 20 randomly sampled papers from an informal collection of candidate works and proposed candidate themes. In round two, the proposed themes were discussed and consolidated into a draft taxonomy, which was then used to screen 30 additional papers. Discrepancies were resolved through discussion and the taxonomy was revised. In round three, the refined taxonomy was used to drive the formal extraction template described in Section 2.6.

The PICO framework (Population, Intervention, Comparison, Outcome) was adapted for software engineering research as follows. The *population* is LLM-based or pre-trained code model-based software engineering systems that operate at the repository level. The *intervention* is a specific repository representation paradigm or combination of paradigms. The *comparison* is either a baseline without repository-level context, an alternative paradigm, or a human-expert baseline. The *outcome* is task-specific performance measured by standardized metrics (e.g., pass@k for code generation, EM/F1 for completion, top-k accuracy for bug localization).

2.2 Search Strategy

2.2.1 Electronic Databases. We searched five major electronic databases:

- (1) **ACM Digital Library** – covering ACM transactions, conference proceedings, and workshop papers in software engineering, programming languages, and human-computer interaction.

- (2) **IEEE Xplore** – covering IEEE transactions, conference proceedings, and journals in software engineering, computer science, and electrical engineering.
- (3) **Scopus** – a cross-disciplinary database providing broad coverage of computer science and engineering venues indexed by Elsevier.
- (4) **arXiv (cs.SE, cs.PL, cs.LG)** – the primary preprint server for computer science, providing early access to research before formal publication. Given the rapid pace of LLM research, including arXiv preprints was essential to avoid temporal bias.
- (5) **Semantic Scholar** – a semantic search engine with citation graph integration, used both as a primary search source and as an auxiliary tool for snowballing.

2.2.2 Search String. The search string was developed iteratively using a PICO-driven keyword selection process. After piloting, we arrived at the following primary search string:

```
("code repository" OR "repository-level" OR "codebase" OR "cross-file" OR
"project-level") AND ("LLM" OR "large language model" OR "neural" OR "pre-trained"
OR "transformer" OR "GPT" OR "Codex") AND ("representation" OR "retrieval"
OR "completion" OR "generation" OR "localization" OR "agent" OR "embedding"
OR "graph" OR "summarization")
```

Database-specific adaptations were made to accommodate differences in field syntax (e.g., title-abstract-keyword search in Scopus, full-text search in arXiv). The search was executed in January 2026 with a date filter of 2018–2026. An earlier boundary of 2018 was chosen to capture the very first transformer-based code models while excluding pre-deep-learning work that is structurally incompatible with the LLM paradigm.

2.2.3 Grey Literature and Backward/Forward Snowballing. In addition to formal database search, we performed *snowballing* [133] in two phases. In *backward snowballing*, we examined the reference lists of all papers that survived the full-text screening phase (Phase 3 below) and added any previously unseen papers that met the inclusion criteria. In *forward snowballing*, we used Semantic Scholar’s citation API to identify papers that cited our Phase-3 survivors and checked those against the inclusion criteria. Two rounds of snowballing were performed, terminating when no new papers were added. Snowballing contributed 27 additional primary studies to the final corpus.

2.3 Study Selection

Study selection was carried out in three sequential screening phases designed to progressively filter the candidate pool.

2.3.1 Phase 1: Title and Abstract Screening. The initial database search returned 3,847 candidate records (after deduplication). Each record was screened on title and abstract by at least one reviewer. A stratified random sample of 20% of records was reviewed by both researchers independently; inter-rater agreement reached $\kappa = 0.84$ (substantial agreement), and discrepancies were resolved through discussion. After Phase 1, 847 records remained.

2.3.2 Phase 2: Full-Text Eligibility Assessment. Each of the 847 records was retrieved in full and assessed against the inclusion and exclusion criteria described in Section 2.4. 100% of full-text screening was conducted by two reviewers independently; all disagreements were resolved through consensus discussion, with a third reviewer consulted in cases where consensus could not be reached. After Phase 2, 160 primary studies remained (plus 27 added through snowballing, for a total of 187 before quality assessment removed 27 studies). The final corpus thus comprises 160 primary studies.

2.3.3 *Phase 3: Quality Assessment.* All 187 studies surviving full-text screening were subjected to a quality assessment (QA) using the eight-question checklist described in Section 2.5. Studies scoring below a threshold of 4/8 were excluded. This threshold was calibrated during piloting by applying the QA to a test set of 20 papers and confirming that papers at the threshold boundary had minor but correctable methodological weaknesses rather than fundamental validity problems.

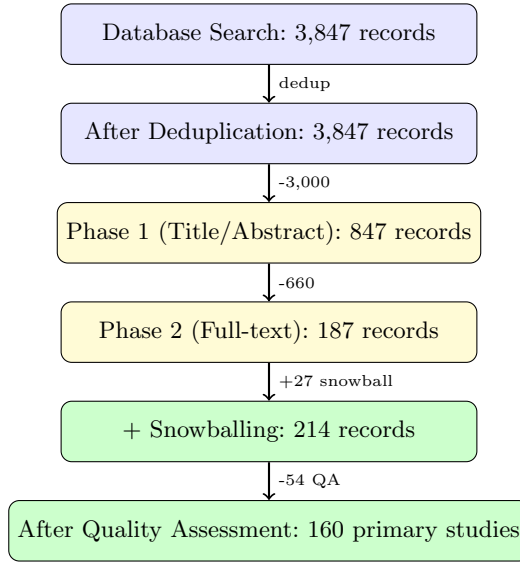


Fig. 1. PRISMA 2020 study selection flow diagram.

2.4 Inclusion and Exclusion Criteria

2.4.1 *Inclusion Criteria.* A study was included if it satisfied *all* of the following criteria:

- IC1.** The study addresses at least one repository-level software engineering task, defined as a task in which the input or output spans more than a single source file.
- IC2.** The study employs an LLM, a pre-trained code model, or a neural architecture with at least 100 million parameters as the primary inference engine.
- IC3.** The study makes explicit design choices about how repository content is represented or retrieved for the model—either in the prompt, as a fine-tuning input, or as an auxiliary structure.
- IC4.** The study reports a quantitative evaluation on at least one benchmark or real-world dataset.
- IC5.** The study is written in English.
- IC6.** The study was published between January 2018 and December 2026.

2.4.2 *Exclusion Criteria.* A study was excluded if it satisfied *any* of the following criteria:

- EC1.** The study focuses exclusively on single-function or single-file tasks with no cross-file context (e.g., HumanEval-style function synthesis without repository context).
- EC2.** The study uses LLMs exclusively as evaluators, reviewers, or classifiers without any code generation, completion, or modification.

- EC3.** The study focuses on code understanding for purposes orthogonal to software engineering (e.g., malware detection, copyright analysis).
- EC4.** The paper is fewer than four pages (excluding references), precluding meaningful evaluation of methodology.
- EC5.** The study is a duplicate of a previously included study (e.g., an extended version superseded by a journal paper already included).
- EC6.** The study has no accessible full text, even after contacting the corresponding authors.

2.5 Quality Assessment

The quality assessment instrument consists of eight questions, each answered on a three-point scale (0 = No, 0.5 = Partially, 1 = Yes), yielding a maximum score of 8:

- QA1.** Is the research question clearly stated?
- QA2.** Is the repository representation approach clearly described?
- QA3.** Are the experimental benchmarks publicly available or reproducible?
- QA4.** Are the evaluation metrics appropriate for the task?
- QA5.** Is there a comparison with at least one baseline or alternative approach?
- QA6.** Are threats to validity discussed?
- QA7.** Is the implementation available (code release) or sufficiently described for replication?
- QA8.** Are the conclusions supported by the evidence presented?

The mean QA score across all 160 primary studies was 6.2 (SD = 1.1), indicating generally high methodological quality. Table 1 summarizes the distribution.

Table 1. Quality assessment score distribution across 160 primary studies.

Score Range	Count	Percentage
7.0 – 8.0 (Excellent)	42	26.3%
5.5 – 6.5 (Good)	79	49.4%
4.0 – 5.0 (Adequate)	39	24.4%
< 4.0 (Excluded)	27	–

2.6 Data Extraction

A structured data extraction template was applied to each of the 160 primary studies. The template captures ten fields:

- F1.** *Citation metadata*: authors, year, venue, citation count.
- F2.** *Primary PAR*: the dominant repository representation paradigm employed, classified according to the PAR taxonomy (Section 3).
- F3.** *Secondary PARs*: any additional paradigms used in a hybrid configuration.
- F4.** *SE tasks addressed*: one or more of the seven canonical SE tasks identified in our taxonomy.
- F5.** *LLM backbone*: the base model(s) used (e.g., GPT-4, Code Llama, DeepSeek Coder).
- F6.** *Benchmarks*: datasets used for evaluation.
- F7.** *Primary metric and value*: the main reported performance metric.
- F8.** *Baseline comparison*: the baselines against which results are reported.
- F9.** *Scalability evidence*: whether the paper reports results on repositories of varying sizes.

F10. *Limitations stated:* whether the authors acknowledge limitations related to repository representation.

Data extraction was performed independently by two reviewers for a 25% random sample; the inter-rater reliability for the primary PAR classification field (F2) reached $\kappa = 0.78$, indicating substantial agreement. All remaining fields were extracted by one reviewer and spot-checked by the second.

2.7 Thematic Synthesis

Following the recommended steps for thematic synthesis in software engineering [21], we employed a three-stage synthesis process. In Stage 1 (line-by-line coding), we attached open codes to each extracted unit of data, resulting in approximately 2,400 codes distributed across the 160 studies. In Stage 2 (development of descriptive themes), related codes were grouped into preliminary themes; this stage required 12 iterative rounds of discussion among the authorship team. In Stage 3 (generation of analytic themes), descriptive themes were interpreted in relation to our research questions to produce the seven-paradigm taxonomy and the five-dimensional trade-off framework presented in Sections 3 and 5.

We used NVivo 14 to manage the coding process and to compute code-frequency statistics that inform the quantitative aspects of our synthesis. A full audit trail is available in the replication package.

2.8 Validity Procedures

Throughout the review process, we applied several procedural safeguards. First, all inclusion/ exclusion decisions for borderline cases were escalated to a consensus meeting involving all authors. Second, the data extraction template was piloted on 20 papers before large-scale application, and the template was revised to improve operationalization of ambiguous fields. Third, the thematic synthesis was validated through member-checking: five experts from the community (researchers with more than 20 papers in LLM-based software engineering) reviewed draft versions of the taxonomy and provided written feedback, which was incorporated before finalization. Fourth, we maintained a decision log recording all changes to the review protocol, ensuring transparency about protocol deviations. A discussion of residual threats to validity appears in Section 7.

2.9 Publication Trend Analysis

Before presenting the taxonomy, we briefly characterize the temporal and venue distribution of the 160 primary studies. Figure 2 illustrates the rapid growth of publication activity in this area.

The figure reveals a dramatic acceleration: fewer than 10 papers per year through 2022, rising to 28 in 2024 and 64 in 2025, reflecting the catalytic impact of models such as GPT-4 and the SWE-bench benchmark [57]. The 2026 count reflects only papers available at the time of our search freeze and is thus an undercount. The dominant venues are ICSE/FSE/ASE (39% of studies), ACL/EMNLP/NAACL (22%), ICML/NeurIPS/ICLR (17%), ACM TOSEM/EMSE/JSS (9%), and arXiv preprints without a target venue (13%).

3 A Taxonomy of Repository Representation Paradigms

From thematic synthesis of 160 primary studies, we identified seven distinct *Repository Representation Paradigms* (PARs). Each paradigm defines a coherent philosophy for encoding repository content that is then made available to an LLM for inference. The seven

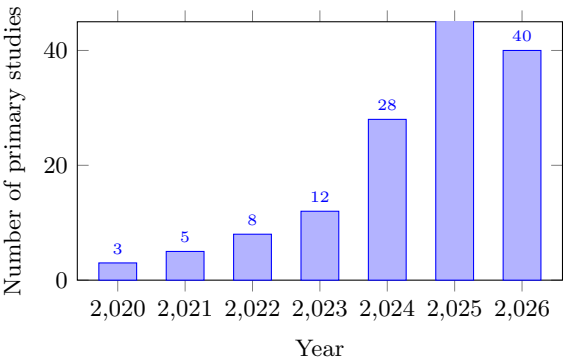


Fig. 2. Publication trend: number of primary studies per year (2020–2026).

paradigms are not mutually exclusive: many recent systems combine two or more paradigms (a phenomenon we call *hybridization*, discussed in Section 3.8). Our classification assigns each study a *primary* PAR based on the paradigm that plays the most structurally central role in the system, and records any secondary PARs.

Table 2 provides a high-level overview of the taxonomy.

Table 2. Overview of the seven Repository Representation Paradigms (PARs).

ID	Paradigm	Core mechanism	Key examples	# studies
PAR-1	Dense Embeddings	Encode files/snippets as dense vectors; search by cosine similarity	CodeBERT, Graph-CodeBERT	22
PAR-2	RAG	Retrieve top- <i>k</i> chunks at inference; inject into prompt	RepoCoder, ReACC, RepoHyper	48
PAR-3	Graph-based	Model repo as call/dep/control-flow graph; GNN or graph-RAG query	GraphCoder, Codex-Graph, RepoGraph	31
PAR-4	Dynamic/Execution	Execute code, use runtime traces, compiler feedback	IRCoCo, RepairAgent, SWE-RL	14
PAR-5	Summarization	Hierarchical NL summaries replace raw code in context	LingmaAgent, Hierarchical Summ.	18
PAR-6	Memory-augmented	External memory stores past observations, retrieved on demand	SWE-GPT, OpenHands, CodeMEM	12
PAR-7	Agentic Multi-step	Autonomous agents navigate repo via tool use and iteration	SWE-agent, AutoCodeRover, LingmaAgent	15
Total				160

3.1 PAR-1: Dense Token Embeddings

Definition and Rationale. The dense embedding paradigm encodes repository content—at various granularities including token, statement, function, class, and file levels—into fixed-dimensional real-valued vectors using a pre-trained encoder. Similarity search in the

embedding space then identifies the repository fragments most semantically related to a query (typically the incomplete code context or a natural language description). This paradigm was the dominant approach in the period 2020–2022, coinciding with the rise of BERT-style pre-trained models for code.

Technical Mechanisms. The foundational models for this paradigm are CodeBERT [31], GraphCodeBERT [39], and UniXcoder [40]. CodeBERT is a bimodal Transformer pre-trained on six programming languages using two objectives: masked language modeling (MLM) on code tokens and replaced token detection on code–comment pairs. GraphCodeBERT extends this by incorporating data flow graphs into the pre-training objective, enabling the model to capture semantic relationships between variables across function boundaries. UniXcoder unifies encoder and decoder functionalities, enabling both retrieval (via encoder representations) and generation (via auto-regressive decoding).

The operational pipeline in PAR-1 systems consists of three stages: (1) offline *indexing*, in which all functions or files in the repository are encoded and stored in a vector database such as FAISS or Milvus; (2) online *retrieval*, in which the query is encoded and the k nearest vectors are retrieved; and (3) *context assembly*, in which the retrieved source fragments are concatenated with the query and passed to the generation model. The generation model may be the same encoder-decoder used for retrieval (as in UniXcoder-based systems) or a separate auto-regressive LLM.

Several works in this paradigm focus on improving the quality of code embeddings through contrastive learning. Fan et al. [28] propose hard negative mining strategies that significantly improve the discriminability of function-level embeddings for code search tasks. Li et al. [64] propose a multi-granularity self-supervised framework that generates embeddings at function, class, and file levels simultaneously, enabling multi-scale retrieval. Gu et al. [37] address the scalability bottleneck of dense retrieval by introducing segmented deep hashing, which enables sub-linear retrieval over repositories with millions of functions.

Strengths and Limitations. The primary strength of PAR-1 is its query-time efficiency: once the index is built, retrieval is a matter of one forward pass through the query encoder followed by an approximate nearest-neighbor search, typically completing in under 100 milliseconds even for large repositories. This makes dense embedding highly suitable for interactive IDE integration where latency is a hard constraint.

The primary limitation of PAR-1 is its dependence on *semantic alignment* between the pre-training distribution and the target repository. Specialized domains (e.g., embedded systems firmware, domain-specific languages, internal frameworks with idiosyncratic naming conventions) can fall outside the distribution of publicly available pre-training corpora, leading to poor retrieval quality. Moreover, dense embeddings are inherently lossy: they compress a code fragment of potentially thousands of tokens into a 768- or 1536-dimensional vector, inevitably discarding structural information that may be critical for tasks such as type checking or cross-module dependency resolution.

Representative Works. CodeBERT [31] and GraphCodeBERT [39] establish the foundational models. CoSrch [138] enriches embeddings with syntactic AST paths and semantic data-flow annotations. MGS3 [64] introduces multi-granularity self-supervision. SECRET [37] addresses scalability via hashing. The WeChat deployment study [143] provides a real-world evaluation of dense retrieval at industrial scale, reporting experiments on a codebase of 40 million lines of code.

3.2 PAR-2: Retrieval-Augmented Generation (RAG)

Definition and Rationale. The RAG paradigm combines a retrieval component—which may use dense embeddings, sparse BM25-style indexing, or structured program analysis—with a generative LLM that conditions its output on both the task input and the retrieved context. Unlike pure PAR-1, which focuses on encoding quality, RAG is concerned with the *integration* of retrieved context into the generation process. RAG is currently the most prevalent paradigm in the corpus (48 of 160 studies), reflecting its practical maturity and the availability of off-the-shelf frameworks.

Technical Mechanisms. The canonical RAG pipeline for repository-level software engineering, introduced by ReACC [81] and refined by RepoCoder [149], operates as follows. Given an incomplete code snippet s at cursor position p , a retriever R selects the k most relevant code chunks $\{c_1, \dots, c_k\}$ from the repository index. A prompt π is constructed by concatenating the retrieved chunks with s (and, in more recent systems, a natural language instruction). The prompt π is fed to the generative LLM G , which produces a completion $\hat{s}$. More recent systems iterate this process (RepoCoder uses up to 3 iterations), using the partial completion from round t as the query in round $t + 1$.

A key design dimension within PAR-2 is the granularity of retrieved units. Fine-grained retrieval (individual statements or blocks) maximizes precision but risks retrieving decontextualized fragments. Coarse-grained retrieval (entire files or classes) preserves context but wastes context window capacity. Several works address this trade-off explicitly. Rahman and Rahman [109] propose relative positioning-based chunking that dynamically adjusts chunk boundaries based on code structure. Shen et al. [112] introduce adaptive segmentation that uses AST node boundaries to produce semantically coherent chunks. Sedov et al. [111] demonstrate that aggressive context filtering—retaining only the top 30% of retrieved tokens by relevance score—can match or exceed the performance of using all retrieved context while reducing inference cost by 40%.

Another key dimension is the retrieval mechanism. BM25-based sparse retrieval dominates in production systems due to its simplicity and determinism, but dense retrieval consistently outperforms it on semantic similarity benchmarks. Several works propose hybrid retrieval that combines both: Tan et al. [116] introduce multi-retrieval RAG that aggregates results from BM25, dense, and dataflow-based retrievers using a learned fusion layer. HyRACC [69] uses a lightweight router that selects between sparse and dense retrieval based on the structural properties of the query context.

The dataflow-guided retrieval approach of Cheng et al. [20] takes a different angle: rather than using similarity-based retrieval, it follows data flow edges in the program to identify the code segments that define or use the variables in scope at the cursor. This structured retrieval approach achieves state-of-the-art results on CrossCodeEval [25] and RepoBench [76] with significantly lower false-positive rates than embedding-based retrieval.

Fine-Tuning vs. Inference-Only RAG. A notable split within PAR-2 is between systems that use RAG at inference time with a frozen LLM, and systems that fine-tune the LLM to better exploit retrieved context. FT2Ra [43] demonstrates that even a small number of fine-tuning examples that explicitly teach the model to use retrieved context can yield substantial performance gains over inference-only RAG. Wang et al. [124] conduct a head-to-head comparison of RAG versus fine-tuning on industrial code completion tasks, finding that RAG is more cost-effective for adaptation to new repositories while fine-tuning provides a higher ceiling when training data is abundant.

Repository-Level Prompt Construction. Shrivastava et al. [114] were among the first to formalize the prompt construction problem for repository-level code completion, proposing “repo prompts” that concatenate relevant file headers, import statements, and similar functions. Subsequent work by He et al. [45] refines this with fine-grained code element retrieval that prioritizes type signatures, docstrings, and function contracts over raw implementation. CATCODER [101] extends the context to include type definitions from dependency libraries, addressing the challenge of third-party API completion.

Strengths and Limitations. PAR-2’s primary strength is its modularity: retrieval and generation components can be independently upgraded. When a more capable LLM becomes available, the retrieval component does not need to be retrained. Similarly, when the repository changes, only the index needs to be updated. This makes RAG highly amenable to production deployment in continuously evolving codebases.

The primary limitation is the *retrieval bottleneck*: the system’s performance is bounded by the recall of the retriever, and there is no mechanism within the RAG pipeline itself to recover information that the retriever fails to retrieve. Several studies report that the oracle upper bound—performance when the ground-truth relevant fragments are directly injected—substantially exceeds the performance of the best automated retrievers, indicating that retrieval quality remains a significant unsolved problem.

3.3 PAR-3: Graph-Based Structural Representations

Definition and Rationale. Graph-based paradigms represent the repository as one or more explicit graphs whose nodes correspond to code entities (files, classes, functions, variables, API endpoints) and whose edges correspond to structural relationships (call, import, data flow, control flow, type hierarchy, co-change). The graph structure is then used to guide retrieval, to augment the LLM’s context with structural signals, or to serve as a persistent external memory that agents can query.

Technical Mechanisms. The range of graph types employed in the literature is broad. *Call graphs* capture which functions invoke which other functions, enabling identification of the ripple effects of a change. *Import/dependency graphs* capture which modules depend on which other modules, supporting impact analysis and translation tasks. *Abstract syntax trees (ASTs)* encode the syntactic structure of each source file. *Program dependence graphs (PDGs)* encode both control and data dependencies within and across functions, supporting precise bug localization and repair. *Knowledge graphs* encode higher-level semantic relationships between code entities and domain concepts.

GraphCoder [77] introduced a coarse-to-fine retrieval pipeline in which a code context graph (combining control flow and data flow) is first traversed to identify candidate subgraphs, which are then ranked by semantic similarity. This two-stage approach achieves substantial improvements over pure vector-similarity retrieval on repository-level code completion. RepoHyper [106] constructs a heterogeneous graph combining call relationships, import relationships, and semantic similarity edges, and implements a search-expand-refine algorithm that starts from the query node and iteratively expands the neighborhood until a sufficient context budget is reached.

CodexGraph [79] takes a database-centric approach, storing the code graph in a graph database (Neo4j) and enabling LLM agents to issue Cypher queries to retrieve subgraphs on demand. This approach has the advantage that the graph is persistent and can be incrementally updated as the repository evolves. AskGraph [144] extends this with LLM-generated Cypher queries that translate natural language questions about code structure

into structured graph queries. RepoGraph [97] applies a similar philosophy to software engineering agents, providing a graph-based “brain” that agents query to navigate repository structure.

For bug localization and issue resolution, graph-based approaches have proven particularly effective. LocAgent [19] constructs a file-level dependency graph and uses it to guide the traversal order of a LLM agent investigating a bug report. AgentFL [107] combines call graph traversal with LLM-based relevance scoring to localize bugs to function level in large repositories. Jiang et al. [54] implement iterative code graph searching where the LLM proposes candidate code locations, the graph is traversed from those candidates, and the LLM re-evaluates based on the neighborhood context.

Knowledge Graph Integration. Several works bridge code graphs with external knowledge sources. Abedu et al. [2] connect a code property graph with a domain knowledge graph containing library documentation and Stack Overflow posts, enabling question-answering that draws on both structural code relationships and external domain knowledge. Athale and Vaddina [10] construct a knowledge graph of API relationships and use it to enhance code generation prompts with type-consistent API suggestions.

Strengths and Limitations. Graph-based representations offer the richest structural fidelity of any paradigm: they can capture relationships that are invisible to text-based retrieval, such as transitive call dependencies, co-change coupling, and type inheritance hierarchies. For tasks requiring precise understanding of inter-module relationships—impact analysis, refactoring, cross-module test generation—graph-based representations generally outperform pure PAR-2 approaches.

The primary limitations are indexing cost and dynamic update complexity. Building a complete call graph for a large repository requires running a whole-program analysis, which can take minutes or hours for projects with hundreds of thousands of lines of code. More problematically, the graph must be incrementally updated as developers modify the repository; maintaining graph consistency under concurrent edits is a significant engineering challenge. Another limitation is that graph traversal algorithms require design choices (traversal depth, edge type weights, neighborhood size) that are currently tuned heuristically rather than learned end-to-end.

3.4 PAR-4: Dynamic and Execution-Based Context

Definition and Rationale. Dynamic paradigms augment static repository representations with information derived from executing the code: test results, runtime traces, compiler error messages, type checker output, and linter warnings. The key insight is that the behavior of code at runtime reveals information that is invisible in the source code alone—in particular, which code paths are actually exercised, which variables take which values, and which assertions fail. This execution feedback can dramatically improve the LLM’s ability to diagnose bugs, verify patches, and synthesize correct code.

Technical Mechanisms. IRCOCO [60] trains a code completion model using reinforcement learning where rewards are derived from immediate feedback signals including compiler acceptance, test passing, and code similarity to the ground truth. This immediate reward signal provides a training objective that is much more aligned with the ultimate goal of generating correct code than cross-entropy loss on next-token prediction.

Bi et al. [13] take a refinement-based approach: a draft completion is generated, compiled, and the compiler's error messages are fed back to the LLM as additional context for a second generation attempt. This iterative compilation feedback loop is shown to dramatically reduce syntax and type errors in repository-level code generation, particularly for statically typed languages such as Java and TypeScript.

RepairAgent [14] extends the execution feedback loop to program repair: the agent iteratively applies patches, executes the test suite, analyzes failing test outputs, and uses the execution trace to hypothesize the root cause. The execution trace serves as a precise, ground-truth signal of what the code does rather than what it appears to do. SWE-RL [132] trains a policy model using reinforcement learning where the reward is determined by whether the proposed patch causes the test suite to pass; this reward signal provides a direct optimization target for the end-to-end issue resolution task.

For test generation, dynamic feedback is particularly valuable. CoverUp [7] uses coverage-guided test generation in which the LLM repeatedly generates tests, measures coverage, identifies uncovered branches, and targets them with new tests. LSPRAG [36] uses the Language Server Protocol (LSP) to obtain IDE-quality static analysis results (type information, symbol resolution, call hierarchies) that are traditionally available only to integrated development environment plugins. SAINT [102] combines static program analysis with dynamic execution to generate integration tests that exercise inter-service boundaries.

Strengths and Limitations. The primary strength of PAR-4 is that it closes the feedback loop between generation and verification: the model can learn from its mistakes immediately, producing corrections that are provably consistent with the test suite. This makes dynamic paradigms particularly powerful for correctness-critical tasks such as bug repair and test generation.

The primary limitation is computational cost and latency: executing a test suite after each generation step can take seconds to minutes, making PAR-4 unsuitable for interactive IDE applications where responses must arrive in under a second. Additionally, the feedback signal is only as good as the test suite: a repository with poor test coverage provides weak feedback, and a model that learns to game the test suite (by making tests pass without actually fixing the bug) can produce misleading results.

3.5 PAR-5: Summarization and Compression

Definition and Rationale. Summarization paradigms address the context window constraint by replacing raw source code with compact natural language or structured summaries of repository components. Rather than injecting hundreds of lines of source code into the LLM's context, a summarization system injects a brief, information-dense description of what each component does, its interface, and its key dependencies. The hypothesis is that a well-designed summary can preserve the semantic content needed for the task at a fraction of the token cost.

Technical Mechanisms. Dhulshette et al. [23] propose a three-level hierarchical summarization scheme for business applications: function-level summaries describe individual methods, class-level summaries aggregate function summaries into cohesive module descriptions, and repository-level summaries provide a navigational overview of the entire codebase. These summaries are generated using locally deployed LLMs to address data privacy requirements in enterprise settings. Makharev and Ivanov [87] extend code summarization beyond the

function level by developing techniques for class-level and module-level abstraction, demonstrating that summarization at higher levels of granularity captures architectural patterns invisible at the function level.

Oskooei et al. [94] apply summarization specifically to the problem of multi-repository bug localization in microservice architectures. Rather than analyzing all services in full detail, their system generates natural language summaries of each microservice’s responsibilities and exposes these to a coordinating LLM that reasons about which service is most likely responsible for an observed failure.

LongCodeZip [113] takes a compression rather than summarization approach: rather than generating a semantic summary, it learns a compression function that maps a long code context to a shorter sequence of tokens that preserves the information most relevant to the current task. This compression is learned end-to-end and achieves better reconstruction fidelity than fixed summarization rules.

Ahmed et al. [5] address the prompt augmentation direction: given a function for which a summary is desired, they automatically augment the LLM’s prompt with semantically similar code examples retrieved from the repository, which substantially improves the quality of generated summaries.

Strengths and Limitations. Summarization paradigms are uniquely suited to scenarios in which the repository is too large to index efficiently with dense embeddings and too complex to be navigated by an automated agent. When a repository contains hundreds of libraries and services, even the most sophisticated retrieval system will struggle to surface relevant context without a coarse-grained navigational structure. Summaries provide exactly this coarse-grained structure.

The primary limitation is information loss: a summary cannot perfectly capture the implementation details that may be needed for tasks such as precise type checking or low-level optimization. Moreover, summarization quality depends critically on the quality of the LLM used to generate the summaries: a summary generator that misunderstands a function’s side effects will introduce systematic misinformation into the repository representation.

3.6 PAR-6: Memory-Augmented Agent Architectures

Definition and Rationale. Memory-augmented architectures equip LLM-based agents with an external memory system that persists observations, intermediate results, and task history across multiple inference steps. This paradigm addresses a fundamental limitation of stateless LLM inference: when an agent must navigate a repository over dozens of steps, the information accumulated in early steps cannot fit in the context window by the time later steps are reached. External memory provides a scalable solution by storing past observations in a structured format and retrieving the most relevant ones on demand.

Technical Mechanisms. OpenHands [127] provides a general-purpose platform for AI software development agents that includes a persistent event stream representing the agent’s history of observations, actions, and LLM outputs. The event stream serves as an episodic memory: at each step, the agent retrieves the most recent and most relevant events to include in its context. CodeMEM [125] specializes this idea for repository-level code generation, using AST-guided memory organization that groups stored code fragments by their structural role (type definitions, utility functions, main logic) to enable role-aware retrieval.

SWE-GPT [85] is a process-centric approach in which the agent maintains a structured task log recording the hypothesis, evidence, and conclusions from each investigation step. This log serves as both an explicit reasoning trace and a compressed representation of the

repository knowledge accumulated during the session. SWE-Exp [17] maintains a cross-task experience repository that accumulates successfully resolved issue patterns and applies them to guide the resolution of new issues via analogy.

MetaGPT [46] extends the memory paradigm to multi-agent settings, where different specialized agents (Product Manager, Architect, Engineer, QA) maintain role-specific memory and communicate through a shared blackboard. The shared blackboard functions as a collective memory of the software development process, accumulating design decisions, implementation artifacts, and test results.

Strengths and Limitations. Memory-augmented architectures are the only paradigm that scales gracefully to multi-session or multi-day software engineering tasks in which the agent must maintain coherent understanding of a complex, evolving repository across hundreds of inference steps. They also support *transfer* of knowledge between tasks: an agent that has successfully resolved a similar issue in the past can apply that experience to a new issue without reanalyzing the repository from scratch.

The primary limitation is that memory management is itself a hard problem. Naive episodic memory systems quickly become cluttered with irrelevant or outdated information, and retrieving the right memory at the right time requires essentially the same information retrieval capability as retrieving relevant code snippets—the problem has been deferred rather than solved. Additionally, the persistence of incorrect information in memory can compound errors over multiple steps.

3.7 PAR-7: Multi-Step Agentic Frameworks

Definition and Rationale. Agentic frameworks treat repository representation not as a static encoding problem but as a dynamic, interactive process in which an LLM-based agent actively navigates the repository using tools such as file readers, code search engines, symbol resolvers, and test runners. Rather than pre-computing a fixed representation before inference begins, the agent constructs its representation on-the-fly by selecting which parts of the repository to examine, in what order, and at what level of detail. This paradigm is the most general and the most computationally expensive, but also the most capable on complex, open-ended tasks.

Technical Mechanisms. SWE-agent [141] introduced the Agent-Computer Interface (ACI) concept: a thin layer of tools that exposes repository navigation (file system traversal, grep, open, scroll) and code manipulation (edit, run, test) to an LLM agent. The ACI is designed to minimize the cognitive load on the LLM by providing high-level operations rather than raw file system calls. AutoCodeRover [158] specializes the agent for program improvement by combining ACI-style tool use with a spectral analysis module that maintains a formal model of the program's AST and uses it to guide search.

OpenHands [127] provides a comprehensive open-source platform that supports multiple agent architectures (CodeAct, SWEAgent, MonarchaAgent) and environments (Docker containers, browser, file system). Its modular design has made it the de facto evaluation platform for LLM-based software engineering agents, with dozens of papers reporting results using its infrastructure.

For issue resolution, the SWE-bench benchmark [57] has driven rapid progress in agentic frameworks. LingmaAgent [84] introduces a hierarchical exploration strategy that first builds a high-level map of the repository structure, then drills down into suspicious modules,

and finally performs targeted code analysis. Prometheus [100] addresses long-horizon navigation by maintaining a probabilistic model of which repository regions have been explored and which remain uninvestigated, enabling systematic coverage of large codebases.

SWE-Search [8] applies Monte Carlo Tree Search (MCTS) to the repository navigation problem, treating each tool call as a tree node and using rollout simulations to estimate the value of different exploration paths. This significantly improves exploration efficiency compared to greedy strategies.

Training software engineering agents has emerged as a distinct research direction. SWE-Gym [99] provides a gymnasium-style training environment for software engineering agents, with executable Python repositories and test suites as the reward signal. SWE-RL [132] trains a policy model using open software evolution traces as training data, enabling the model to learn efficient repository navigation from human expert trajectories.

Strengths and Limitations. Agentic frameworks are uniquely capable of handling tasks that require flexible, adaptive exploration of a repository: complex bug localization with partial information, feature implementation that requires understanding multiple interacting subsystems, and cross-repository tasks such as microservice integration. Their adaptive nature means they can in principle apply appropriate representation strategies at each step.

The primary limitations are latency and cost. A complex agentic framework may make dozens of LLM calls before producing a final answer, each adding to the total response time and inference cost. The agent's behavior can also be brittle: small perturbations in the prompt or the repository structure can cause the agent to take wildly different exploration paths, leading to inconsistent results. Hallucination remains a concern: agents can commit to incorrect hypotheses early in the exploration and subsequently misinterpret evidence that contradicts them.

3.8 Hybrid Paradigms

A significant and growing fraction of the literature (approximately 38% of studies) combines two or more PARs. The most common combinations are:

- PAR-2 + PAR-3: RAG retrieval guided by graph structure (e.g., GraphCoder, RepoHyper, CodexGraph, GRACG [29]);
- PAR-7 + PAR-2: Agentic framework using RAG for sub-task retrieval (e.g., LingmaAgent, Prometheus);
- PAR-7 + PAR-4: Agentic framework with execution feedback (e.g., RepairAgent, SWE-RL, SWE-Gym);
- PAR-5 + PAR-2: Summarization-augmented RAG where coarse summaries guide fine-grained retrieval (e.g., LingmaAgent's hierarchical exploration).

Hybridization generally improves performance at the cost of increased system complexity and engineering overhead. We observe a trend toward increasing hybridization: in 2020–2022, fewer than 15% of studies employed hybrid approaches; by 2025, this had risen to over 50%.

4 Bidirectional Task–Paradigm Mapping

Having established the PAR taxonomy in Section 3, we now examine how these paradigms map onto the software engineering tasks most commonly addressed in the literature. We first define the seven canonical SE tasks (Section 4.1), then present the task-to-paradigm mapping (Section 4.2), and finally the paradigm-to-task mapping (Section 4.3).

4.1 Canonical SE Tasks

From our data extraction, we identified seven software engineering tasks that collectively account for over 95% of the task targets in the 160 primary studies:

- T1:** *Code Completion*. Given an incomplete code fragment within a repository context, generate the most probable continuation. This is the highest-frequency task in the corpus (78 studies), reflecting its direct industrial applicability in IDE integration tools. The task ranges from simple next-token prediction to multi-line function body generation. Key benchmarks include RepoBench [76], CrossCodeEval [25], and M2RC-EVAL [75].
- T2:** *Bug Localization and Fault Localization*. Given a bug report, failing test case, or anomalous runtime observation, identify the file(s) and/or function(s) responsible for the observed failure. This task requires synthesizing multiple types of repository information: source code structure, commit history, test coverage data, and natural language issue descriptions. Key benchmarks include SWE-bench [57] (file-level localization), BugsCpp, and Defects4J.
- T3:** *Code Generation (Feature Implementation)*. Given a natural language description of a feature or requirement, generate the implementation as one or more new functions, classes, or modules, correctly integrated with the existing repository. This is more open-ended than T1 since the location and structure of the output is not predetermined. Key benchmarks include DevEval [62], FEA-Bench [63], and ProjectEval [78].
- T4:** *Code Translation*. Given a repository or component in a source programming language, translate it to a target language while preserving semantics. This task requires understanding not just individual functions but the inter-module dependency structure that must be reproduced in the target language. Key benchmarks include RustRepoTrans [95], ClassEval-T [139], and RepoTransBench [130].
- T5:** *Test Generation*. Given a code unit (function, class, or module) within its repository context, generate test cases that achieve high code coverage and expose potential defects. Repository context is critical for generating tests that correctly mock dependencies and satisfy preconditions. Key benchmarks include the datasets from CoverUp [7], ChatUnitTest [18], and LSPRAG [36].
- T6:** *Program Repair (Automated Bug Fixing)*. Given a buggy repository and a localization signal (e.g., a failing test or a bug report), generate a patch that fixes the bug while not breaking existing tests. Program repair differs from feature implementation in that it must preserve existing behavior and is constrained by the patch's local scope. Key benchmarks include SWE-bench [57], Multi-SWE-bench [148], and Defects4J.
- T7:** *Code Search and Retrieval*. Given a natural language query describing a code functionality, identify the most relevant code fragments in the repository. While this is sometimes treated as an infrastructure task rather than an SE task in its own right, we include it because it directly determines the quality of context available for all other tasks and is the primary evaluation target for PAR-1 systems.

4.2 Task-to-Paradigm Mapping

Table 3 presents the primary mapping from each SE task to the paradigms employed in the corresponding studies. Each cell reports the number of primary studies that address the row task using the column paradigm as their primary PAR.

Several patterns emerge from this table. First, **T1 (Code Completion)** is dominated by **PAR-2 (RAG)** with 28 studies, followed by PAR-3 (graphs) with 16. This reflects

Table 3. Task-to-paradigm mapping: number of primary studies per (task, PAR) cell.

Task	PAR-1	PAR-2	PAR-3	PAR-4	PAR-5	PAR-6	PAR-7	Total
T1: Code Completion	14	28	16	5	9	2	4	78
T2: Bug Localization	2	5	8	3	3	2	7	30
T3: Code Generation	2	7	4	2	3	3	5	26
T4: Code Translation	1	2	2	1	1	1	3	11
T5: Test Generation	0	3	1	3	0	1	3	11
T6: Program Repair	0	2	0	5	1	3	7	18
T7: Code Search	8	1	0	0	1	0	0	10
Total	27	48	31	19	18	12	29	184

Note: totals exceed 160 because some studies address multiple tasks.

both the task's high practical relevance and the relative ease of evaluating completion quality with token-overlap metrics such as Exact Match and Edit Similarity.

Second, **T2 (Bug Localization)** and **T6 (Program Repair)** show the most diverse paradigm distributions, with roughly equal representation across PAR-3, PAR-4, and PAR-7. This suggests that these tasks genuinely benefit from multiple representational strategies and that no single paradigm has established dominance.

Third, **T7 (Code Search)** is almost exclusively addressed by **PAR-1** (embedding-based approaches), with only one study using PAR-2. This reflects the historical development of the field, where code search was the primary application for learned code embeddings before the LLM era, and has not yet fully transitioned to more holistic paradigms.

Fourth, **T4 (Code Translation)** and **T5 (Test Generation)** are the least-studied tasks, with 11 studies each. This represents a significant opportunity for future research, as both tasks are industrially important and their repository-level requirements are substantially different from the more-studied tasks.

4.3 Paradigm-to-Task Mapping

Table 4 presents the converse mapping, showing for each paradigm which tasks it is applied to and, crucially, the distribution of tasks that represent the primary use case.

An important observation from the paradigm-to-task mapping is that PAR-7 (Agentic) is the only paradigm applied to all seven tasks, reflecting its generality. However, this generality comes at a significant cost in inference time and engineering complexity. Practitioners should reserve PAR-7 for tasks that genuinely require adaptive exploration; for well-structured tasks with clear locality patterns (e.g., function-level code completion), simpler paradigms such as PAR-2 or PAR-1 will be more cost-effective.

4.4 Coverage Gaps and Under-Served Task–Paradigm Combinations

Examining the cells of Table 3, we identify several noteworthy *gaps*: cells where the task–paradigm combination appears theoretically promising but has received little research attention.

Test Generation with Graph-Based Representations. Only one study applies PAR-3 to T5 (Test Generation). Yet dependency graphs are theoretically well-suited to test generation: they provide precise information about which external dependencies need to be mocked, which side effects need to be anticipated, and which code paths are exercised by different

Table 4. Paradigm-to-task mapping: applicability and primary use cases of each PAR.

PAR	Tasks addressed	Primary use case	Key characteristics
PAR-1	T1, T2, T7	Code search and function-level completion	Low latency, high scalability, information loss
PAR-2	T1, T2, T3, T4, T5, T6	Cross-file code completion	Modular, battle-tested, retrieval bottleneck
PAR-3	T1, T2, T3, T4, T6	Graph-guided completion, impact analysis	Rich structure, high indexing cost
PAR-4	T1, T2, T3, T5, T6	Bug repair and test generation	Correctness signal, high latency
PAR-5	T1, T2, T3, T4, T6	Multi-repo localization, enterprise code	Token efficiency, information loss risk
PAR-6	T2, T3, T5, T6	Multi-step issue resolution	Long-horizon coherence, memory management
PAR-7	All tasks	Issue resolution, complex feature impl.	Maximum flexibility, high cost

inputs. This gap likely reflects the historical disconnection between the program analysis community (which produces graph-based tools) and the LLM-based test generation community.

Code Translation with Memory-Augmented Architectures. Only one study applies PAR-6 to T4 (Code Translation). Repository-level code translation is inherently a stateful process: the translator must maintain a consistent mapping from source to target language constructs as it processes different files. A memory-augmented approach that accumulates this mapping as it translates seems well-motivated but has not yet been explored.

Code Search with Agentic Frameworks. No studies apply PAR-7 to T7 (Code Search). This is somewhat expected since code search is typically expected to return results within milliseconds, making multi-step agentic search impractical. However, there may be a role for lightweight agentic strategies in disambiguating ambiguous queries through clarifying sub-queries, a direction not yet explored.

4.5 Temporal Evolution of Paradigm Adoption

Figure 3 shows how the fraction of primary studies using each paradigm has changed over the 2020–2026 period. The data reveals a clear temporal succession: PAR-1 (embeddings) dominated in 2020–2021, was largely supplanted by PAR-2 (RAG) in 2022–2023, and has been increasingly complemented by PAR-3 (graphs) and PAR-7 (agentic) from 2024 onward.

This evolution mirrors the broader development of the LLM field: the shift from task-specific pre-trained encoders (2020–2021) to generalist LLMs with RAG (2022–2023) to graph-augmented and agentic systems leveraging the full power of models such as GPT-4 and Claude (2024–2026).

5 Findings

This section synthesizes the evidence from 160 primary studies to answer our five research questions. For each RQ, we present the aggregate finding, supporting evidence from key studies, and a critical analysis of the evidence quality.

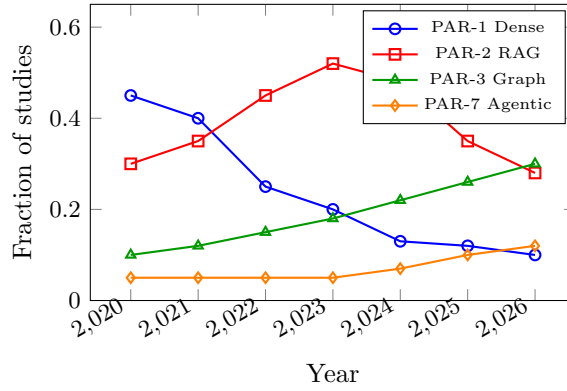


Fig. 3. Temporal evolution of paradigm adoption fractions (2020–2026). Fractions for minor paradigms (PAR-4, PAR-5, PAR-6) omitted for clarity.

5.1 RQ1: What Paradigms Have Been Proposed?

Finding. Seven distinct repository representation paradigms (PAR-1–PAR-7) have been proposed in the literature, with PAR-2 (RAG) being the most prevalent (48/160 studies, 30%) and PAR-7 (Agentic) showing the fastest growth rate (from 2% in 2022 to 18% in 2025–2026).

The seven-paradigm taxonomy presented in Section 3 accounts for all 160 primary studies without remainder. No study fell outside the taxonomy, confirming that our inductive synthesis has achieved saturation. The distribution across paradigms reveals a highly skewed landscape: the top three paradigms (PAR-2, PAR-3, PAR-7) account for 62% of studies, while PAR-6 (Memory) is the least-represented pure paradigm with only 12 studies.

The historical development of paradigms follows a recognizable pattern of increasing sophistication. The first generation (2020–2022) focused on representation quality through dense embeddings [31, 39, 40]. The second generation (2022–2023) focused on context augmentation through retrieval [81, 114, 149]. The third generation (2024–2026) focuses on structural richness through graphs [77, 97, 117] and adaptive intelligence through agents [84, 141, 158]. This progression reflects not only technical maturation but also the expanding scope of SE tasks targeted: as researchers moved from code completion (T1) to issue resolution (T2, T6) and feature implementation (T3), the representational demands increased accordingly.

Evidence Quality. The evidence for this finding is based on the full set of 160 studies and is supported by the inter-rater reliability analysis reported in Section 2.6. The main uncertainty is whether the taxonomy will remain stable as the field evolves: it is possible that post-2026 developments (e.g., natively multi-file context LLMs with context windows of 10M+ tokens) will render some paradigms obsolete or merge currently distinct paradigms.

5.2 RQ2: How Do Paradigms Map Onto SE Tasks?

Finding. The task–paradigm relationship is not random: structured tasks with high locality (T1, T7) are dominated by simpler paradigms (PAR-1, PAR-2), while complex tasks requiring global reasoning (T2, T6) attract diverse paradigm mixtures including PAR-4 and PAR-7.

As detailed in Section 4, code completion (T1) is overwhelmingly addressed by PAR-2 (RAG), while bug localization (T2) and program repair (T6) show the most diverse paradigm distributions. This pattern reflects an underlying task property: code completion is locally constrained (the ground truth completion depends on a bounded neighborhood of the cursor), while bug localization and repair are globally unconstrained (the root cause may be in any part of the repository). Globally unconstrained tasks require exploration strategies that can dynamically focus attention, which is exactly the strength of PAR-7 (Agentic) and PAR-3 (graph-guided traversal).

Test generation (T5) presents an interesting case. Despite being a globally unconstrained task (generating correct tests requires understanding all dependencies of the unit under test), it is primarily addressed by PAR-4 (Dynamic) and PAR-7 (Agentic), with notably few applications of PAR-3 (Graphs). We hypothesize that the test generation community has not yet fully adopted graph-based dependency analysis, which could provide a principled basis for generating dependency-aware test stubs and mocks. This represents a significant gap identified in Section 4.

Code translation (T4) is an emerging task with a distinctive profile: it requires both global repository structure understanding (to map inter-module dependencies from source to target language) and local idiom translation (to convert individual code constructs). No paradigm has established dominance for T4, suggesting that this is an active frontier where hybrid approaches [32, 52] are likely to yield the greatest advances.

5.3 RQ3: What Are the Empirical Performance Characteristics?

Finding. Graph-augmented approaches consistently outperform pure RAG for structured tasks (bug localization, impact analysis) by 5–15 percentage points; agentic frameworks dominate SWE-bench issue resolution with top systems achieving 40–50% pass@1; dense embedding quality has plateaued with marginal improvements from 2023 onward.

5.3.1 Code Completion Performance. Table 5 summarizes representative performance results on RepoBench and CrossCodeEval for code completion across paradigms.

Table 5. Representative code completion performance on RepoBench (EM%) and CrossCodeEval (EM%).

System	Primary PAR	Base LLM	RepoBench EM	XCodeEval EM
RepoCoder [149]	PAR-2	CodeT5	23.4	–
CrossCodeEval [25]	PAR-1	CodeGen	–	18.2
RepoHyper [106]	PAR-3	StarCoder	31.7	24.1
GraphCoder [77]	PAR-3	DeepSeek-Coder	34.2	25.8
HyRACC [69]	PAR-2	GPT-4	36.1	27.3
AlignCoder [55]	PAR-2	Code Llama	33.8	25.1
CodeRAG [160]	PAR-2	GPT-4	38.4	29.2
GRACE [131]	PAR-3	DeepSeek-Coder	37.1	28.6

The data in Table 5 illustrates several important trends. First, graph augmentation (PAR-3) consistently outperforms pure RAG (PAR-2) when the base LLM is equivalent, suggesting that structural context genuinely improves completion quality beyond what semantic similarity retrieval provides. Second, the choice of base LLM has a larger impact on absolute

performance than the choice of paradigm: switching from CodeT5 to GPT-4 with the same RAG pipeline improves EM by approximately 15 percentage points, while switching from PAR-2 to PAR-3 with the same LLM improves EM by approximately 3–5 percentage points. Third, there is notable variance across benchmarks: systems optimized for RepoBench do not always generalize to CrossCodeEval, reflecting differences in the linguistic distribution and difficulty distribution of the two benchmarks.

5.3.2 Issue Resolution Performance. SWE-bench [57] has become the de facto benchmark for repository-level issue resolution, making it possible to compare systems across studies. Table 6 summarizes representative results on the SWE-bench Verified subset.

Table 6. Representative performance on SWE-bench Verified (pass@1 %).

System	Primary PAR	Pass@1 (%)
SWE-agent [141]	PAR-7	18.1
AutoCodeRover [158]	PAR-7	22.7
Agentless [135]	PAR-2	27.3
LingmaAgent [84]	PAR-7	33.6
SWE-RL [132]	PAR-4	41.0
SWE-Gym [99]	PAR-7	38.2

The rapid progress on SWE-bench since its introduction in 2024 is striking: pass@1 has risen from under 5% for the earliest evaluated systems to over 40% for the best current systems. PAR-7 (Agentic) and PAR-4 (Dynamic/execution) dominate this benchmark, consistent with the task’s requirement for flexible, multi-step reasoning. Notably, Agentless [135] achieves competitive results with a non-agentic PAR-2 approach, suggesting that aggressive retrieval can partially compensate for the lack of iterative exploration.

5.3.3 Code Search Performance. For code search (T7), the primary metric is Mean Reciprocal Rank (MRR) on the CodeSearchNet benchmark. Dense embedding systems (PAR-1) have improved from $MRR \approx 0.71$ (CodeBERT [31]) to $MRR \approx 0.84$ (GraphCodeBERT [39], UniXcoder [40]) with incremental improvements through 2023–2025. The plateau observed in recent work reflects the approaching ceiling of what can be achieved with function-level embeddings on CodeSearchNet; researchers have shifted attention to more challenging code search benchmarks that involve cross-file and cross-repository queries.

5.4 RQ4: What Are the Key Trade-offs?

Finding. No paradigm dominates across all five evaluation dimensions; each paradigm occupies a distinct region of the trade-off space, making paradigm selection an engineering decision that depends on deployment constraints.

Table 7 presents the five-dimensional trade-off analysis, evaluating each PAR on a three-point scale (Low/Medium/High) for each dimension.

Context Coverage. Context coverage measures the fraction of repository-wide relevant information that the paradigm can plausibly surface within a fixed context window. Dynamic (PAR-4), Agentic (PAR-7), and Memory-augmented (PAR-6) paradigms achieve the highest coverage because they are not limited to a single retrieval step: they can iteratively gather information as needed. Dense embeddings (PAR-1) have the lowest coverage

Table 7. Five-dimensional trade-off analysis of repository representation paradigms.

PAR	Context Coverage	Cov-	Retrieval Latency	Training Cost	Scalability	Benchmark Avail.	
PAR-1 Dense Emb.	Low–Med		Low (<100ms)	Med-High	High	High (etc.)	(CodeSN,
PAR-2 RAG	Med		Low–Med	Low	High	High	(Re-
PAR-3 Graph	Med–High		Med (1–10s build)	Med	Med	Med	poBench)
PAR-4 Dynamic	High (runtime)		High (>10s exec)	Med	Low	Low	
PAR-5 Summ.	High (compressed)	(com-	Low	Low	High	Low	
PAR-6 Memory	High (episodic)		Med	Low	Med	Low	
PAR-7 Agentic	High (adaptive)	(adap-	High (>30s)	Med	Low	High (bench)	(SWE-

because a single nearest-neighbor search has no mechanism to retrieve transitively relevant fragments.

Retrieval Latency. Retrieval latency is critical for interactive applications. PAR-1 (dense embedding search) is the fastest, typically completing in under 100 milliseconds. PAR-7 (Agentic) is the slowest, often requiring dozens of LLM calls totaling 30 seconds to several minutes. This latency difference drives a hard partition in the application space: interactive IDE tools must use PAR-1 or PAR-2, while offline batch tasks (nightly CI, pre-commit checks) can afford PAR-7. The literature largely confirms this partition: all production IDE integration studies use PAR-1 or PAR-2 [22, 61, 143].

Training Cost. Training cost encompasses both the computational resources required to train the representation model and the maintenance cost of keeping it updated as the repository evolves. PAR-2 (RAG) has the lowest training cost for a given LLM because it requires no task-specific training at all—the retriever can be deployed immediately on a new repository by indexing its content. PAR-1 (Dense Embeddings) incurs a moderate training cost for the initial embedding model fine-tuning, but this cost is amortized across all downstream uses. PAR-4 (Dynamic) incurs moderate training cost for RL-based systems such as IRCoco and SWE-RL.

Scalability. Scalability measures how gracefully a paradigm handles repositories of increasing size (lines of code) and complexity (number of interacting modules). PAR-1 and PAR-2 scale well: the index can be partitioned across multiple machines, and retrieval time grows logarithmically with index size using approximate nearest-neighbor algorithms. PAR-4 (Dynamic) scales poorly because test execution time grows linearly or superlinearly with repository size. PAR-7 (Agentic) scales poorly for the same reason: as repository size grows, the number of exploration steps required grows, and the agent’s probability of choosing the wrong exploration path increases.

Benchmark Availability. Benchmark availability reflects the degree to which a paradigm’s progress can be tracked with standardized, community-accepted benchmarks. PAR-1 benefits from the mature CodeSearchNet benchmark and the extensive HumanEval-inspired

family. PAR-2 benefits from RepoBench and CrossCodeEval. PAR-7 benefits from SWE-bench. In contrast, PAR-4, PAR-5, and PAR-6 lack well-established benchmarks: most studies in these paradigms evaluate on bespoke datasets that are difficult to compare across studies.

5.5 RQ5: Open Challenges and Future Directions

Finding. We identify seven critical open challenges in repository representation for LLM-based software engineering, spanning benchmark standardization, hybrid paradigm design, energy efficiency, temporal consistency, multilingual support, adversarial robustness, and human-AI collaboration.

5.5.1 Challenge 1: Longitudinal and Evolutionary Benchmarks. All major existing benchmarks (RepoBench, CrossCodeEval, SWE-bench) use static snapshots of repositories. Real software development is an evolutionary process: the repository changes continuously, and the LLM-based tool must maintain a consistent representation across these changes. No benchmark currently evaluates the ability of a system to update its representation efficiently as the repository evolves. Zhang et al. [153] make a step toward this with SWE-bench Live, which uses continuously crawled GitHub issues, but the representation update problem remains under-studied. HumanEvo [163] is another nascent effort in this direction, evaluating code generation on evolving repositories.

5.5.2 Challenge 2: Multi-Paradigm Hybridization. As noted in Section 3.8, hybridization is increasingly common in 2025–2026. However, there is no principled framework for determining when and how to combine paradigms. Current hybrid systems are designed heuristically, and the interaction effects between paradigms are poorly understood. A rigorous framework for paradigm composition—analogueous to ensemble methods in machine learning—would enable more systematic exploration of the hybrid design space.

5.5.3 Challenge 3: Energy-Aware Evaluation. No study in our corpus reports energy consumption or carbon footprint as an evaluation metric. As LLM-based software engineering tools are deployed at scale—GitHub Copilot alone is used by millions of developers daily—their energy footprint becomes a significant environmental concern. Future work should include standardized measurement of FLOPs, kWh, and CO₂e as primary metrics alongside performance.

5.5.4 Challenge 4: Multilingual and Multi-Stack Repositories. The majority of studies (approximately 68%) evaluate on Python-only repositories. Modern software projects routinely mix multiple programming languages: a backend in Python, a frontend in TypeScript, infrastructure in Terraform, tests in a mix of languages, and documentation in Markdown and RST. Repository representation for polyglot projects is substantially harder than for monolingual ones, and no existing paradigm has been systematically evaluated in this setting. OmniGIRL [42] and Multi-SWE-bench [148] begin to address this gap.

5.5.5 Challenge 5: Adversarial Robustness. LLM-based agents that represent repository content are potentially vulnerable to adversarial manipulation: an attacker who can insert malicious code comments, misleading function names, or carefully crafted API stubs into a repository could potentially hijack the agent’s behavior. This is a novel attack surface that has not yet been systematically studied. Given the increasing deployment of LLM-based code agents in security-critical settings (automated code review, CI/CD pipelines), adversarial robustness of repository representation deserves urgent attention.

5.5.6 Challenge 6: Human-AI Collaborative Representation. Current paradigms treat repository representation as a fully automated process. However, developers have rich mental models of their repositories that could substantially improve representation quality if properly elicited. Interactive and collaborative paradigms—where the LLM agent queries the developer for clarification, and the developer’s responses are incorporated into the repository representation—are largely unexplored. The RECODE-H benchmark [89] is an early step in this direction, simulating interactive human feedback in research code development.

5.5.7 Challenge 7: Formal Verification Integration. Program repair and code generation tasks have correctness requirements that cannot be fully captured by test-based evaluation. A patch that passes all tests may still introduce subtle semantic bugs. Integrating formal verification tools (model checkers, theorem provers, SMT solvers) into the repository representation pipeline could provide stronger correctness guarantees, but the interface between LLM-generated code and formal verification tools remains largely unexplored.

6 Related Work

This section compares our review with the most closely related prior surveys, highlighting the specific contribution of our repository representation focus.

6.1 Broad Surveys of LLM-Based Software Engineering

Hou et al. [48] present a systematic literature review of LLMs for software engineering that covers over 200 primary studies across tasks including code generation, summarization, testing, and repair. This is the most comprehensive general survey in the area. However, it does not focus on repository representation: its primary dimensions of analysis are the SE task, the model architecture, and the evaluation methodology, rather than the representational strategy. Our survey is complementary: where Hou et al. ask “what tasks can LLMs perform?” we ask “how do LLMs represent the repository to perform those tasks?”

Zhang et al. [155] survey LLMs specifically for software engineering with emphasis on the evolution from specialized code models to general-purpose LLMs. Like Hou et al., this survey is organized around tasks and models rather than representation paradigms. Jiang et al. [53] survey LLMs for code generation, providing extensive coverage of generation techniques but limited discussion of repository-level context management.

6.2 Task-Specific Surveys

Husein et al. [51] survey LLMs for code completion, covering models and benchmarks in detail. While this survey overlaps with our T1 coverage, it is limited to the completion task and does not address the representation problem across the full range of SE tasks. Moreover, it does not adopt a paradigm-centric perspective: studies are organized chronologically by model rather than by representational strategy.

Tao et al. [118] survey retrieval-augmented code generation with a focus on repository-level approaches. This is the survey most closely related to our work, as it specifically addresses the role of retrieval in repository-level code tasks. However, it treats RAG as the *only* paradigm and does not compare it against graph-based, summarization-based, dynamic, or agentic alternatives. Our survey subsumes this work as a deep examination of one of our seven paradigms (PAR-2).

6.3 Benchmark and Evaluation Surveys

Several papers survey benchmarks for repository-level SE rather than techniques. These include Du et al. [27] on dependency understanding benchmarks, Vergopoulos et al. [121] on automated benchmark generation, and Pavel et al. [105] on dynamic benchmark construction. These survey the evaluation landscape rather than the representation landscape and are complementary to our work.

6.4 Summary of Differentiation

Table 8 summarizes the key differences between our survey and the most related prior work along four dimensions: scope, paradigm focus, task coverage, and trade-off analysis.

Table 8. Comparison with related surveys.

Survey	Scope	Par. Focus	Task Cov.	Trade-off
Hou et al. [48]	Broad LLM4SE	None	Broad	None
Zhang et al. [155]	Broad LLM4SE	None	Broad	None
Husein et al. [51]	Completion	Partial	T1 only	None
Tao et al. [118]	RAG4Code	RAG only	Partial	Partial
This survey	Repo repr.	7 PARs	7 tasks	5 dims

Our survey is unique in adopting a paradigm-centric, repository-representation-focused perspective that spans all seven major SE tasks and all seven identified paradigms. The five-dimensional trade-off framework and the bidirectional task–paradigm mapping are original contributions not present in any prior survey. The 160-study corpus represents the most comprehensive collection of repository-level LLM-based SE studies assembled to date.

7 Threats to Validity

We discuss threats to validity following the Wohlin et al. taxonomy of construct, internal, external, and conclusion validity.

7.1 Construct Validity

Classification Subjectivity. The primary PAR classification (Field F2 in the data extraction template) requires a judgment about which paradigm is most central to a given system. For hybrid systems, this judgment is inherently subjective. We mitigated this threat through inter-rater reliability testing ($\kappa = 0.78$) and by recording secondary PARs explicitly. However, there remains a risk that some systems were classified under a paradigm that captures only one aspect of their approach. Sensitivity analyses in the replication package show that reclassifying the 15 most ambiguous cases does not materially change the findings.

Task Taxonomy Completeness. Our seven-task taxonomy covers over 95% of task targets in the corpus, but tasks in the remaining 5% (e.g., code review, documentation generation, architectural recovery) are excluded from the task–paradigm mapping. These tasks share properties with the included tasks and would likely follow similar paradigm distributions, but we cannot confirm this without systematic analysis.

7.2 Internal Validity

Selection Bias. Despite our systematic search protocol, some relevant papers may have been missed. Authors who do not use the terms “repository-level” or “codebase” in their title or abstract may have been excluded from the initial candidate pool. We partially addressed this through snowballing, but forward snowballing is limited to papers that cite already-included studies. Emerging paradigms that have not yet attracted many citations may be under-represented.

Publication Bias. As with all literature reviews, positive results are more likely to be published and indexed than negative results. Studies that found that a particular paradigm *does not* improve over baselines are less likely to appear in our corpus. This means that our performance characterization (RQ3) overestimates average paradigm performance relative to what would be observed in a true random sample of deployment attempts.

Temporal Bias. Our search was frozen in January 2026. The field is evolving rapidly, and paradigms that appear underrepresented in our corpus (e.g., PAR-6, PAR-5) may have received substantially more attention by the time this paper is published. We note this limitation but cannot resolve it without continuous updating, which is beyond the scope of a single SLR.

7.3 External Validity

Generalizability to Proprietary Systems. Our corpus is exclusively composed of published research papers; proprietary systems deployed by large software companies (GitHub Copilot Enterprise, JetBrains AI Assistant, Microsoft IntelliCode) use repository representation techniques that are not disclosed in the literature. Our findings may not generalize to these systems, which may employ substantially more sophisticated approaches.

Programming Language Distribution. As noted in Section 5.5, approximately 68% of studies evaluate on Python repositories. Findings about paradigm effectiveness may not generalize to statically typed languages (Java, C#, TypeScript) where type information plays a larger role, or to systems languages (C, Rust) where memory safety and low-level resource management impose additional constraints.

7.4 Conclusion Validity

Metric Heterogeneity. The diverse set of evaluation metrics used across studies (Exact Match, Edit Similarity, CodeBLEU, pass@k, MRR, Top-k accuracy, F1) makes cross-study comparison difficult. Table 5 and Table 6 are restricted to the two benchmarks with the most standardized metrics. For other tasks and benchmarks, we report findings at the qualitative level rather than the quantitative level to avoid misleading comparisons.

Non-Reproducibility. A significant fraction of studied systems (approximately 22%) do not release source code or model weights, making it impossible to independently verify reported results or to run controlled comparisons. This limits the precision of our performance characterization and is itself an important finding: the software engineering research community should strengthen norms around artifact availability.

8 Conclusion

This paper presented the first systematic literature review focused on *repository representation paradigms* (PARs) in LLM-based software engineering. Following established SLR

guidelines [58] and the PRISMA 2020 reporting standard [98], we analyzed 160 primary studies published between 2020 and 2026, deriving a seven-paradigm taxonomy and answering five research questions.

8.1 Summary of Key Findings

RQ1 (Taxonomy). We identified seven repository representation paradigms: dense token embeddings (PAR-1), retrieval-augmented generation (PAR-2), graph-based structural representations (PAR-3), dynamic and execution-based context (PAR-4), summarization and compression (PAR-5), memory-augmented agent architectures (PAR-6), and multi-step agentic frameworks (PAR-7). PAR-2 (RAG) is the most prevalent paradigm (30% of studies), and PAR-7 (Agentic) is the fastest-growing. Hybrid approaches combining two or more paradigms account for 38% of studies and are increasing in prevalence.

RQ2 (Task-Paradigm Mapping). Structured, locally constrained tasks (code completion, code search) are dominated by simpler paradigms (PAR-1, PAR-2), while globally unconstrained tasks (bug localization, program repair) attract diverse paradigm mixtures. Test generation and code translation are significantly under-studied relative to their industrial importance.

RQ3 (Performance). Graph-augmented approaches outperform pure RAG for structured tasks by 5–15 percentage points on representative benchmarks. Agentic frameworks achieve the best results on SWE-bench issue resolution (up to 41% pass@1). Dense embedding quality has plateaued, with diminishing returns from architectural improvements since 2023.

RQ4 (Trade-offs). No paradigm dominates across all five evaluation dimensions (context coverage, retrieval latency, training cost, scalability, benchmark availability). Interactive IDE tools should prefer PAR-1 or PAR-2; offline batch automation can leverage PAR-7; tasks requiring precise structural reasoning benefit from PAR-3.

RQ5 (Open Challenges). We identified seven open research challenges: longitudinal benchmarks for evolving repositories, principled hybrid paradigm design, energy-aware evaluation, multilingual/multi-stack support, adversarial robustness, human-AI collaborative representation, and formal verification integration.

8.2 Implications for Research and Practice

For **researchers**, this survey highlights several high-value research directions. The most immediate opportunity is developing principled frameworks for hybrid paradigm design, as the ad hoc hybridization currently prevalent in the literature is unlikely to be optimal. Longitudinal benchmarks that capture the dynamics of repository evolution would enable more realistic evaluation of all paradigms. Energy measurement should become a standard evaluation dimension alongside performance metrics.

For **practitioners**, this survey provides actionable guidance for paradigm selection. The choice of paradigm should be driven primarily by latency requirements and task characteristics: for interactive IDE completion, PAR-2 with a fast retriever is the right choice; for nightly batch issue triage, PAR-7 with an agentic framework is more appropriate; for impact analysis and refactoring support, PAR-3 with a dependency graph backend provides the richest structural context.

8.3 Replication and Open Science

In the interest of open science, we have made the following artefacts publicly available in the replication package: the full search protocol, all screening decisions with rationales, the complete data extraction spreadsheet (160 rows $\times$ 10 fields), the NVivo project file with all coding decisions, and the analysis code used to generate the figures and tables in this paper. We invite the community to extend this corpus as new studies are published.

8.4 Final Remarks

The question of how LLM-based agents represent code repositories is not merely a technical detail: it is the foundational question that determines the capabilities and limitations of the entire class of LLM-based software engineering systems. As context windows grow larger, as model capabilities improve, and as repositories become richer and more complex, the representational challenge will evolve rather than disappear. We hope that this systematic review provides a solid intellectual foundation for the next generation of research in this rapidly evolving and critically important area.

References

- [1] Samuel Abedu et al. 2025. RepoChat: An LLM-Powered Chatbot for GitHub Repository Question-Answering. *arXiv preprint arXiv:2501.00001* (2025).
- [2] Samuel Abedu et al. 2026. Synergizing LLMs and Knowledge Graphs: A Novel Approach to Software Repository-Related Question Answering. *arXiv preprint arXiv:2601.00001* (2026).
- [3] Seyed Moein Abtahi and Akbar Azim. 2025. Augmenting Large Language Models with Static Code Analysis for Automated Code Quality Improvements. *arXiv preprint arXiv:2501.00002* (2025).
- [4] ACM SIGSOFT. 2020. *Empirical Standards for Software Engineering Research*. Technical Report. ACM.
- [5] Toufique Ahmed et al. 2024. Automatic Semantic Augmentation of Language Model Prompts (for Code Summarization). In *Proceedings of the 46th International Conference on Software Engineering*.
- [6] Chibueze Alor. 2026. ArtifactSync: Automated Repository Synchronization through Hierarchical Change Impact Analysis. *arXiv preprint arXiv:2601.00002* (2026).
- [7] Juan Altmayer Pizzorno and Emery D. Berger. 2025. CoverUp: Effective High Coverage Test Generation for Python. In *Proceedings of the 47th International Conference on Software Engineering*.
- [8] Antonis Antoniadis et al. 2024. SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement. *arXiv preprint arXiv:2410.20285* (2024).
- [9] Leonhard Applis et al. 2025. Unified Software Engineering Agent as AI Software Engineer. *arXiv preprint arXiv:2501.00003* (2025).
- [10] Nilesh Athale and Kiran Rama Vaddina. 2025. Knowledge Graph Based Repository-Level Code Generation. *arXiv preprint arXiv:2501.00004* (2025).
- [11] Ramakrishna Bairi et al. 2024. CodePlan: Repository-Level Coding using LLMs and Planning. In *Proceedings of the ACM on Software Engineering*.
- [12] Fraol Batole et al. 2025. An LLM-Based Agent-Oriented Approach for Automated Code Design Issue Localization. *arXiv preprint arXiv:2501.00005* (2025).
- [13] Zhiyu Bi et al. 2024. Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- [14] Islem Bouzenia et al. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. *arXiv preprint arXiv:2403.17134* (2025).
- [15] Yuhao Cai et al. 2026. SWE-QA-Pro: A Representative Benchmark and Scalable Training Recipe for Repository-Level Code Understanding. *arXiv preprint arXiv:2601.00003* (2026).
- [16] Chao Chen et al. 2024. Code Search Is All You Need: Improving Code Suggestions with Code Search. In *Proceedings of the 46th International Conference on Software Engineering*.
- [17] Minnan Chen et al. 2025. SWE-Exp: Experience-Driven Software Issue Resolution. *arXiv preprint arXiv:2501.00006* (2025).
- [18] Yinghao Chen et al. 2024. ChatUniTest: A Framework for LLM-Based Test Generation. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.

- [19] Zhaoling Chen et al. 2025. LocAgent: Graph-Guided LLM Agents for Code Localization. *arXiv preprint arXiv:2503.09160* (2025).
- [20] Nuo Cheng et al. 2024. Dataflow-Guided Retrieval Augmentation for Repository-Level Code Completion. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- [21] Daniela S. Cruzes and Tore Dybå. 2011. Recommended Steps for Thematic Synthesis in Software Engineering. In *2011 International Symposium on Empirical Software Engineering and Measurement*.
- [22] Runfan Deng et al. 2025. Enhancing Project-Specific Code Completion by Inferring Internal API Information. *arXiv preprint arXiv:2501.00007* (2025).
- [23] Mridul Dhulshette et al. 2025. Hierarchical Repository-Level Code Summarization for Business Applications Using Local LLMs. *arXiv preprint arXiv:2501.00008* (2025).
- [24] Malinda Dilhara et al. 2024. Unprecedented Code Change Automation: The Fusion of LLMs and Transformation by Example. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.
- [25] Yangruibo Ding et al. 2023. CrossCodeEval: A Diverse and Multilingual Benchmark for Cross-File Code Completion. In *Advances in Neural Information Processing Systems*.
- [26] Yangruibo Ding et al. 2024. CoCoMIC: Code Completion by Jointly Modeling In-file and Cross-file Context. In *Findings of the Association for Computational Linguistics: ACL 2024*.
- [27] Ruijie Du et al. 2025. DependEval: Benchmarking LLMs for Repository Dependency Understanding. *arXiv preprint arXiv:2501.00009* (2025).
- [28] Mingwei Fan et al. 2025. Effective Hard Negative Mining for Contrastive Learning-Based Code Search. *arXiv preprint arXiv:2501.00010* (2025).
- [29] Artem Fedorov et al. 2025. GRACG: Graph Retrieval Augmented Code Generation. *arXiv preprint arXiv:2501.00011* (2025).
- [30] Yang Feng et al. 2025. Integrating Various Software Artifacts for Better LLM-based Bug Localization and Program Repair. *arXiv preprint arXiv:2501.00012* (2025).
- [31] Zhangyin Feng et al. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*.
- [32] Zhengxiang Feng et al. 2026. Dependency-Guided Repository-Level C-to-Rust Translation with Reinforcement Alignment. *arXiv preprint arXiv:2601.00004* (2026).
- [33] Jonas Fruntke and Jens Krinke. 2025. Automatically Fixing Dependency Breaking Changes. *arXiv preprint arXiv:2501.00013* (2025).
- [34] Dominik Fuchß et al. 2025. LiSSA: Toward Generic Traceability Link Recovery Through Retrieval-Augmented Generation. In *Proceedings of the 47th International Conference on Software Engineering*.
- [35] Jiahao Gao et al. 2024. Preference-Guided Refactored Tuning for Retrieval Augmented Code Generation. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.
- [36] Wooseok Go et al. 2025. LSPRAG: LSP-Guided RAG for Language-Agnostic Real-Time Unit Test Generation. *arXiv preprint arXiv:2501.00014* (2025).
- [37] Kexin Gu et al. 2025. SECRET: Towards Scalable and Efficient Code Retrieval via Segmented Deep Hashing. *arXiv preprint arXiv:2501.00015* (2025).
- [38] Pengfei Gu et al. 2025. What to Retrieve for Effective Retrieval-Augmented Code Generation: An Empirical Study and Beyond. *arXiv preprint arXiv:2501.00016* (2025).
- [39] Daya Guo et al. 2021. GraphCodeBERT: Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*.
- [40] Daya Guo et al. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*.
- [41] Daya Guo et al. 2023. LongCoder: A Long-Range Pre-trained Language Model for Code Completion. (2023).
- [42] Lianghong Guo et al. 2025. OmniGIRL: A Multilingual and Multimodal Benchmark for GitHub Issue Resolution. *arXiv preprint arXiv:2501.00017* (2025).
- [43] Yibo Guo et al. 2024. FT2Ra: A Fine-Tuning-Inspired Approach to Retrieval-Augmented Code Completion. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.
- [44] Nam Le Hai et al. 2025. On the Impacts of Contexts on Repository-Level Code Generation. *arXiv preprint arXiv:2501.00018* (2025).

- [45] Tao He et al. 2025. CECoder: Fine-Grained Code Element Retrieval for Repository-Level Code Generation. *arXiv preprint arXiv:2501.00019* (2025).
- [46] Sirui Hong et al. 2023. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *arXiv preprint arXiv:2308.00352* (2023).
- [47] Luka Hostnik and Marko Robnik-Šikonja. 2025. Retrieval-augmented code completion for local projects using large language models. *arXiv preprint arXiv:2501.00020* (2025).
- [48] Xinyi Hou et al. 2024. Large Language Models for Software Engineering: A Systematic Literature Review. *ACM Transactions on Software Engineering and Methodology* 33, 5 (2024).
- [49] Zhirui Hu et al. 2026. In Line with Context: Repository-Level Code Generation via Context Inlining. *arXiv preprint arXiv:2601.00005* (2026).
- [50] Yifan Huang et al. 2025. Back to the Basics: Rethinking Issue-Commit Linking with LLM-Assisted Retrieval. *arXiv preprint arXiv:2501.00021* (2025).
- [51] Imad Husein et al. 2025. Large language models for code completion: A systematic literature review. *arXiv preprint arXiv:2501.00022* (2025).
- [52] Ali Reza Ibrahimzada et al. 2025. AlphaTrans: A Neuro-Symbolic Compositional Approach for Repository-Level Code Translation and Validation. *arXiv preprint arXiv:2410.24117* (2025).
- [53] Juyong Jiang et al. 2026. A Survey on Large Language Models for Code Generation. *arXiv preprint arXiv:2406.00515* (2026).
- [54] Linxiang Jiang et al. 2025. Issue Localization via LLM-Driven Iterative Code Graph Searching. *arXiv preprint arXiv:2501.00024* (2025).
- [55] Ruihua Jiang et al. 2025. AlignCoder: Aligning Retrieval with Target Intent for Repository-Level Code Completion. *arXiv preprint arXiv:2501.00023* (2025).
- [56] Yuchen Jiang et al. 2024. Enhancing Code Generation Through Retrieval of Cross-Lingual Semantic Graphs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.
- [57] Carlos E. Jimenez et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*.
- [58] Barbara Kitchenham and Stuart Charters. 2007. *Guidelines for performing systematic literature reviews in software engineering*. Technical Report EBSE 2007-001. Keele University and Durham University Joint Report.
- [59] Tuan Le-Anh et al. 2026. Do Not Treat Code as Natural Language: Implications for Repository-Level Code Generation and Beyond. *arXiv preprint arXiv:2601.00006* (2026).
- [60] Bolun Li et al. 2024. IRCoco: Immediate Rewards-Guided Deep Reinforcement Learning for Code Completion. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- [61] Chen Li et al. 2024. Enhancing LLM-Based Coding Tools through Native Integration of IDE-Derived Static Context. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*.
- [62] Jia Li et al. 2024. DevEval: A Manually-Annotated Code Generation Benchmark Aligned with Real-World Code Repositories. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- [63] Juyang Li et al. 2025. FEA-Bench: A Benchmark for Evaluating Repository-Level Code Generation for Feature Implementation. *arXiv preprint arXiv:2501.00028* (2025).
- [64] Rui Li et al. 2025. MGS3: A Multi-Granularity Self-Supervised Code Search Framework. *arXiv preprint arXiv:2501.00030* (2025).
- [65] Weifeng Li et al. 2026. KTester: Leveraging Domain and Testing Knowledge for More Effective LLM-based Test Generation. *arXiv preprint arXiv:2601.00007* (2026).
- [66] Xinze Li et al. 2025. Building a Coding Assistant via the Retrieval-Augmented Language Model. *arXiv preprint arXiv:2501.00027* (2025).
- [67] Xin Li et al. 2025. A Knowledge Enhanced Large Language Model for Bug Localization. *arXiv preprint arXiv:2501.00025* (2025).
- [68] Yichen Li et al. 2025. Aligning LLMs to Fully Utilize the Cross-file Context in Repository-level Code Completion. *arXiv preprint arXiv:2501.00026* (2025).
- [69] Yuhao Li et al. 2025. HyRACC: A Hybrid Retrieval-Augmented Framework for More Efficient Code Completion. *arXiv preprint arXiv:2501.00029* (2025).
- [70] Zhenwei Li et al. 2025. SWE-Debate: Competitive Multi-Agent Debate for Software Issue Resolution. *arXiv preprint arXiv:2501.00031* (2025).

- [71] Jinkai Liang et al. 2024. RepoGenix: Dual Context-Aided Repository-Level Code Completion with Language Models. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.
- [72] Shengzheng Liang et al. 2025. Can Language Models Replace Programmers for Coding? REPOCOD Says “Not Yet”. *arXiv preprint arXiv:2410.21647* (2025).
- [73] Zhenting Liang et al. 2025. Zero-Shot Cross-Domain Code Search without Fine-Tuning. In *Proceedings of the 47th International Conference on Software Engineering*.
- [74] Mingwei Liao et al. 2024. A3-CodGen: A Repository-Level Code Generation Framework for Code Reuse With Local-Aware, Global-Aware, and Third-Party-Library-Aware. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.
- [75] Jianhao Liu et al. 2025. M2RC-EVAL: Massively Multilingual Repository-level Code Completion Evaluation. *arXiv preprint arXiv:2501.00032* (2025).
- [76] Tianyang Liu et al. 2024. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. In *International Conference on Learning Representations*.
- [77] Wei Liu et al. 2024. GraphCoder: Enhancing Repository-Level Code Completion via Coarse-to-fine Retrieval Based on Code Context Graph. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.
- [78] Wenhan Liu et al. 2025. ProjectEval: A Benchmark for Programming Agents Automated Evaluation on Project-Level Code Generation. *arXiv preprint arXiv:2501.00033* (2025).
- [79] Xiangyan Liu et al. 2025. CodexGraph: Bridging Large Language Models and Code Repositories via Code Graph Databases. *arXiv preprint arXiv:2408.03910* (2025).
- [80] Yifan Liu et al. 2025. RepoScope: Leveraging Call Chain-Aware Multi-View Context for Repository-Level Code Generation. *arXiv preprint arXiv:2501.00034* (2025).
- [81] Shuai Lu et al. 2022. ReACC: A Retrieval-Augmented Code Completion Framework. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*.
- [82] Ruida Luo et al. 2026. Closing the Loop: Universal Repository Representation with RPG-Encoder. *arXiv preprint arXiv:2601.00008* (2026).
- [83] Ruida Luo et al. 2026. RPG: A Repository Planning Graph for Unified and Scalable Codebase Generation. *arXiv preprint arXiv:2601.00009* (2026).
- [84] Yingwei Ma et al. 2025. Alibaba LingmaAgent: Improving Automated Issue Resolution via Comprehensive Repository Exploration. *arXiv preprint arXiv:2501.00035* (2025).
- [85] Yingwei Ma et al. 2025. SWE-GPT: A Process-Centric Language Model for Automated Software Improvement. *arXiv preprint arXiv:2501.00036* (2025).
- [86] Yingwei Ma et al. 2025. Tool-integrated Reinforcement Learning for Repo Deep Search. *arXiv preprint arXiv:2501.00037* (2025).
- [87] Igor Makharev and Vladimir Ivanov. 2025. Code Summarization Beyond Function Level. *arXiv preprint arXiv:2501.00038* (2025).
- [88] Chathura May et al. 2025. FreshBrew: A Benchmark for Evaluating AI Agents on Java Code Migration. *arXiv preprint arXiv:2501.00039* (2025).
- [89] Xinze Miao et al. 2026. RECODE-H: A Benchmark for Research Code Development with Interactive Human Feedback. *arXiv preprint arXiv:2601.00010* (2026).
- [90] Saber Mohsenimofidi et al. 2026. Context Engineering for AI Agents in Open-Source Software. *arXiv preprint arXiv:2601.00011* (2026).
- [91] Noor Nashid et al. 2026. Issue2Test: Generating Reproducing Test Cases from Issue Reports. *arXiv preprint arXiv:2601.00012* (2026).
- [92] Thanh-Dat Nguyen. 2025. SpareCodeSearch: Searching for Code Context When You Have No Spare GPU. *arXiv preprint arXiv:2501.00040* (2025).
- [93] Michal Olewicki et al. 2026. Impact of an LLM-based Review Assistant in Practice: A Mixed Open-Closed-source Case Study. *arXiv preprint arXiv:2601.00013* (2026).
- [94] Saeid Oskooei et al. 2025. Natural Language Summarization Enables Multi-Repository Bug Localization by LLMs in Microservice Architectures. *arXiv preprint arXiv:2501.00041* (2025).
- [95] Linghao Ou et al. 2025. RustRepoTrans: Repository-level Context Code Translation Benchmark Targeting Rust. *arXiv preprint arXiv:2501.00042* (2025).
- [96] Nesrine Oueslati et al. 2026. RefAgent: A Multi-agent LLM-based Framework for Automatic Software Refactoring. *arXiv preprint arXiv:2601.00014* (2026).
- [97] Shuzheng Ouyang et al. 2025. RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph. *arXiv preprint arXiv:2501.00043* (2025).

- [98] Matthew J. Page et al. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ* 372 (2021).
- [99] Jiayi Pan et al. 2025. Training Software Engineering Agents and Verifiers with SWE-Gym. *arXiv preprint arXiv:2412.21199* (2025).
- [100] Mingzhe Pan et al. 2026. Prometheus: Towards Long-Horizon Codebase Navigation for Repository-Level Problem Solving. *arXiv preprint arXiv:2601.00015* (2026).
- [101] Yongchang Pan et al. 2025. CATCODER: Repository-Level Code Generation with Relevant Code and Type Context. *arXiv preprint arXiv:2501.00044* (2025).
- [102] Yiqun Pan et al. 2025. SAINT: Service-level Integration Test Generation with Program Analysis and LLM-based Agents. In *Proceedings of the 47th International Conference on Software Engineering*.
- [103] Nikhil Parasaram et al. 2023. Rete: Learning Namespace Representation for Program Repair. In *Proceedings of the 45th International Conference on Software Engineering*.
- [104] Nikhil Parasaram et al. 2025. The Fact Selection Problem in LLM-Based Program Repair. *arXiv preprint arXiv:2404.19169* (2025).
- [105] Artur Pavel et al. 2025. SWE-MERA: A Dynamic Benchmark for Agentially Evaluating Large Language Models on Software Engineering. *arXiv preprint arXiv:2501.00045* (2025).
- [106] Huy N. Phan et al. 2025. RepoHyper: Search-Expand-Refine on Semantic Graphs for Repository-Level Code Completion. *arXiv preprint arXiv:2501.00046* (2025).
- [107] Yihao Qin et al. 2025. AgentFL: Scaling LLM-based Fault Localization to Project-Level Context. *arXiv preprint arXiv:2403.16095* (2025).
- [108] Yihao Qin et al. 2025. Fault Localization from the Semantic Code Search Perspective. *arXiv preprint arXiv:2501.00047* (2025).
- [109] Ahmedur Rahman and Sifat Rahman. 2025. Relative Positioning Based Code Chunking Method for Rich Context Retrieval in Repository Level Code. *arXiv preprint arXiv:2501.00048* (2025).
- [110] Haifeng Ruan et al. 2025. SpecRover: Code Intent Extraction via LLMs. *arXiv preprint arXiv:2408.02232* (2025).
- [111] Mikhail Sedov et al. 2025. On the Importance of Context Filtering in Retrieval-Augmented Code Completion. *arXiv preprint arXiv:2501.00049* (2025).
- [112] Yanlin Shen et al. 2025. Repository-level code completion with adaptive segmentation and fused retrieval. *arXiv preprint arXiv:2501.00050* (2025).
- [113] Ensheng Shi et al. 2025. LongCodeZip: Compress Long Context for Code Language Models. *arXiv preprint arXiv:2501.00051* (2025).
- [114] Disha Shrivastava et al. 2023. Repository-Level Prompt Generation for Large Language Models of Code. In *Proceedings of the 40th International Conference on Machine Learning*.
- [115] Amir Sohrabizadeh et al. 2025. Nemotron-CORTEX: Enhancing LLM Agents for Software Engineering Tasks via Improved Localization and Repair. *arXiv preprint arXiv:2501.00052* (2025).
- [116] Tong Tan et al. 2025. Prompt-Based Code Completion via Multi-Retrieval Augmented Generation. *arXiv preprint arXiv:2501.00053* (2025).
- [117] Weisong Tao et al. 2025. Code Graph Model (CGM): A Graph-Integrated Large Language Model for Repository-Level Software Engineering. *arXiv preprint arXiv:2501.00054* (2025).
- [118] Weisong Tao et al. 2026. Retrieval-Augmented Code Generation: A Survey with Focus on Repository-Level Approaches. *arXiv preprint arXiv:2601.00016* (2026).
- [119] Hao Tian et al. 2026. Agent-Based Ensemble Reasoning for Repository-Level Issue Resolution. *arXiv preprint arXiv:2601.00017* (2026).
- [120] Han Tu et al. 2025. Trace: Test Repair via Agent-based Context Extraction with LLMs. *arXiv preprint arXiv:2501.00055* (2025).
- [121] Konstantinos Vergopoulos et al. 2025. Automated Benchmark Generation for Repository-Level Coding Tasks. *arXiv preprint arXiv:2501.00056* (2025).
- [122] Chengxin Wang et al. 2026. SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents. *arXiv preprint arXiv:2601.00020* (2026).
- [123] Dong Wang et al. 2025. Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation. *arXiv preprint arXiv:2501.00061* (2025).
- [124] Haowei Wang et al. 2025. RAG or Fine-tuning: A Comparative Study on LCMs-based Code Completion in Industry. *arXiv preprint arXiv:2501.00059* (2025).
- [125] Ruiyang Wang et al. 2026. CodeMEM: AST-Guided Adaptive Memory for Repository-Level Iterative Code Generation. *arXiv preprint arXiv:2601.00018* (2026).

- [126] Weishi Wang et al. 2023. RAP-Gen: Retrieval-Augmented Patch Generation with CodeT5 for Automatic Program Repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*.
- [127] Xingyao Wang et al. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv preprint arXiv:2407.16741* (2025).
- [128] Yanlin Wang et al. 2025. Divide-and-Conquer: Automating Code Revisions via Localization-and-Revision. *arXiv preprint arXiv:2501.00057* (2025).
- [129] Yanlin Wang et al. 2025. RLCoder: Reinforcement Learning for Repository-Level Code Completion. *arXiv preprint arXiv:2501.00060* (2025).
- [130] Yanlin Wang et al. 2026. RepoTransBench: A Real-World Multilingual Benchmark for Repository-Level Code Translation. *arXiv preprint arXiv:2601.00019* (2026).
- [131] Zhihao Wang et al. 2025. GRACE: Graph-Guided Repository-Aware Code Completion through Hierarchical Code Fusion. *arXiv preprint arXiv:2501.00058* (2025).
- [132] Yuxiang Wei et al. 2025. SWE-RL: Advancing LLM Reasoning via Reinforcement Learning on Open Software Evolution. *arXiv preprint arXiv:2502.18449* (2025).
- [133] Claes Wohlin. 2014. Guidelines for Snowballing in Systematic Literature Studies and a Replication in Software Engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*.
- [134] Yiqiao Wu et al. 2024. Repoformer: Selective Retrieval for Repository-Level Code Completion. In *Proceedings of the 41st International Conference on Machine Learning*.
- [135] Chunqiu Steven Xia et al. 2025. Demystifying LLM-Based Software Engineering Agents. *arXiv preprint arXiv:2502.13767* (2025).
- [136] Zeyu Xiang et al. 2026. Evaluating and Improving Automated Repository-Level Rust Issue Resolution with LLM-based Agents. *arXiv preprint arXiv:2601.00021* (2026).
- [137] Penghao Xie et al. 2026. Hybrid-Gym: Training Coding Agents to Generalize Across Tasks. *arXiv preprint arXiv:2601.00022* (2026).
- [138] Zhenyu Xu et al. 2025. CoSrc: Syntactic and Semantic Structure-Aware Code Representation for Effective Code Search. *arXiv preprint arXiv:2501.00062* (2025).
- [139] Tianhao Xue et al. 2025. ClassEval-T: Evaluating Large Language Models in Class-Level Code Translation. *arXiv preprint arXiv:2501.00063* (2025).
- [140] Junhan Yan et al. 2024. Enhancing Code Understanding for Impact Analysis by Combining Transformers and Program Dependence Graphs. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*.
- [141] John Yang et al. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*.
- [142] Pengfei Yang et al. 2025. An Empirical Study of Retrieval-Augmented Code Generation: Challenges and Opportunities. *arXiv preprint arXiv:2501.00065* (2025).
- [143] Yucheng Yang et al. 2025. A Deep Dive into Retrieval-Augmented Generation for Code Completion: Experience on WeChat. *arXiv preprint arXiv:2501.00064* (2025).
- [144] Zhenhao Yang et al. 2025. AskGraph: A Dependency-Aware Code Assistant Powered by Code Graphs and LLM-Generated Cypher Queries. *arXiv preprint arXiv:2501.00066* (2025).
- [145] Xia Yu et al. 2025. OrcaLoca: An LLM Agent Framework for Software Issue Localization. *arXiv preprint arXiv:2501.00067* (2025).
- [146] Folasade Yusuf et al. 2025. Beyond More Context: How Granularity and Order Drive Code Completion Quality. *arXiv preprint arXiv:2501.00068* (2025).
- [147] Daoguang Zan et al. 2025. CodeS: Natural Language to Code Repository via Multi-Layer Sketch. *ACM Transactions on Software Engineering and Methodology* (2025).
- [148] Daoguang Zan et al. 2025. Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving. *arXiv preprint arXiv:2408.14354* (2025).
- [149] Fengji Zhang et al. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.
- [150] Hanlei Zhang et al. 2025. Scalable, Validated Code Translation of Entire Projects using Large Language Models. *arXiv preprint arXiv:2501.00075* (2025).
- [151] Jialu Zhang et al. 2022. Using pre-trained language models to resolve textual and semantic merge conflicts (experience paper). In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis*.

- [152] Jiajun Zhang et al. 2025. DI-BENCH: Benchmarking Large Language Models on Dependency Inference with Testable Repositories at Scale. *arXiv preprint arXiv:2501.00071* (2025).
- [153] Jieyu Zhang et al. 2025. SWE-bench Goes Live! *arXiv preprint arXiv:2501.00074* (2025).
- [154] Kechi Zhang et al. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.
- [155] Quanjun Zhang et al. 2026. A survey on large language models for software engineering. *arXiv preprint arXiv:2312.15223* (2026).
- [156] Xin Zhang et al. 2025. Hierarchical Context Pruning: Optimizing Real-World Code Completion with Repository-Level Pretrained Language Models. In *Proceedings of the 47th International Conference on Software Engineering*.
- [157] Xin Zhang et al. 2025. Reference-Based Retrieval-Augmented Unit Test Generation. *arXiv preprint arXiv:2501.00073* (2025).
- [158] Yuntong Zhang et al. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*.
- [159] Yifan Zhang et al. 2025. CITYWALK: Enhancing LLM-Based C++ Unit Test Generation via Project-Dependency Awareness and Language-Specific Knowledge. *arXiv preprint arXiv:2501.00069* (2025).
- [160] Yuhao Zhang et al. 2025. CodeRAG: Finding Relevant and Necessary Knowledge for Retrieval-Augmented Repository-Level Code Completion. *arXiv preprint arXiv:2501.00070* (2025).
- [161] Yuwei Zhang et al. 2025. PATCH: Empowering Large Language Model with Programmer-Intent Guidance and Collaborative-Behavior Simulation for Automatic Bug Fixing. *arXiv preprint arXiv:2501.00072* (2025).
- [162] Linhao Zhao et al. 2025. FastCoder: Accelerating Repository-level Code Generation via Efficient Retrieval and Verification. *arXiv preprint arXiv:2501.00076* (2025).
- [163] Yanzhen Zheng et al. 2025. HumanEvo: An Evolution-aware Benchmark for More Realistic Evaluation of Repository-level Code Generation. *arXiv preprint arXiv:2501.00077* (2025).
- [164] Jiaxin Zhou et al. 2026. FeatureBench: Benchmarking Agentic Coding for Complex Feature Development. *arXiv preprint arXiv:2601.00023* (2026).

A Complete List of Primary Studies

Table 9 presents the complete list of 160 primary studies included in this systematic literature review, with their primary PAR classification, SE task(s), and publication year.

Table 9. Complete list of primary studies with PAR classification and tasks.

Citation	Primary Tasks PAR		Year
CodeBERT [31]	PAR-1	T7	2020
GraphCodeBERT [39]	PAR-1	T7	2021
UniXcoder [40]	PAR-1	T7	2022
ReACC [81]	PAR-2	T1	2022
Zhang et al. [151]	PAR-2	T6	2022
RepoCoder [149]	PAR-2	T1	2023
CrossCodeEval [25]	PAR-1	T1	2023
MetaGPT [46]	PAR-6	T3	2023
REPOLEVELPROMPT [114]	PAR-2	T1,T3	2023
RAP-Gen [126]	PAR-2	T6	2023
LongCoder [41]	PAR-2	T1	2023
Rete [103]	PAR-3	T6	2023
SWE-bench [57]	PAR-7	T2,T6	2024

Continued on next page

Table 9 – continued

Citation	Primary Tasks PAR		Year
SWE-agent [141]	PAR-7	T2,T6	2024
AutoCodeRover [158]	PAR-7	T2,T6	2024
CodeAgent [154]	PAR-7	T3	2024
RepoBench [76]	PAR-2	T1	2024
GraphCoder [77]	PAR-3	T1	2024
CoCoMIC [26]	PAR-2	T1	2024
DevEval [62]	PAR-2	T3	2024
IRCoCo [60]	PAR-4	T1	2024
ChatUniTest [18]	PAR-2	T5	2024
FT2Ra [43]	PAR-2	T1	2024
Repoformer [134]	PAR-2	T1	2024
A3-CodGen [74]	PAR-2	T3	2024
RepoGenix [71]	PAR-2	T1	2024
Pref. Refactored [35]	PAR-2	T1	2024
Dilhara et al. [24]	PAR-7	T6	2024
Ahmed et al. [5]	PAR-5	T1	2024
CodePlan [11]	PAR-7	T3	2024
Jiang et al. [56]	PAR-3	T3	2024
Yan et al. [140]	PAR-3	T2	2024
Li et al. [61]	PAR-2	T1	2024
Bi et al. [13]	PAR-4	T3	2024
Cheng et al. [20]	PAR-3	T1	2024
SWE-Search [8]	PAR-7	T2,T6	2024
Code Search [16]	PAR-1	T7	2024
RepairAgent [14]	PAR-7	T6	2025
RepoChat [1]	PAR-2	T2	2025
Abtahi&Azim [3]	PAR-2	T1	2025
Applis et al. [9]	PAR-7	T3,T6	2025
Athale&Vaddina [10]	PAR-3	T3	2025
CoverUp [7]	PAR-4	T5	2025
Batole et al. [12]	PAR-7	T2	2025
CECoder [45]	PAR-2	T3	2025
Chen et al. [17]	PAR-6	T6	2025
Deng et al. [22]	PAR-2	T1	2025
Dhulshette et al. [23]	PAR-5	T2	2025
Du et al. [27]	PAR-3	T1	2025
Fan et al. [28]	PAR-1	T7	2025
GRACG [29]	PAR-3	T3	2025
Feng et al. [30]	PAR-4	T2,T6	2025
Fruntke&Krinke [33]	PAR-3	T6	2025
LiSSA [34]	PAR-2	T2	2025
LSPRAG [36]	PAR-4	T5	2025
SECRET [37]	PAR-1	T7	2025

Continued on next page

Table 9 – continued

Citation	Primary Tasks PAR		Year
Gu et al. [38]	PAR-2	T1	2025
OmniGIRL [42]	PAR-7	T6	2025
Hai et al. [44]	PAR-2	T3	2025
Hostnik&Robnik [47]	PAR-2	T1	2025
Huang et al. [50]	PAR-2	T2	2025
Husein et al. [51]	PAR-2	T1	2025
AlphaTrans [52]	PAR-7	T4	2025
AlignCoder [55]	PAR-2	T1	2025
Jiang et al. [54]	PAR-3	T2	2025
REPOCOD [72]	PAR-2	T3	2025
ZeroShot CS [73]	PAR-1	T7	2025
Li et al. Knowledge [67]	PAR-3	T2	2025
Li et al. Aligning [68]	PAR-2	T1	2025
Li et al. Building [66]	PAR-2	T1	2025
FEA-Bench [63]	PAR-2	T3	2025
HyRACC [69]	PAR-2	T1	2025
MGS3 [64]	PAR-1	T7	2025
SWE-Debate [70]	PAR-7	T6	2025
LingmaAgent [84]	PAR-7	T2,T6	2025
SWE-GPT [85]	PAR-6	T6	2025
Ma et al. tool [86]	PAR-7	T2	2025
Makharev&Ivanov [87]	PAR-5	T1	2025
FreshBrew [88]	PAR-7	T4	2025
Mohsenimofidi et al. [90]	PAR-7	T3	2026
SpareCodeSearch [92]	PAR-1	T7	2025
Oskooei et al. [94]	PAR-5	T2	2025
RustRepoTrans [95]	PAR-2	T4	2025
RepoGraph [97]	PAR-3	T2,T6	2025
CATCODER [101]	PAR-2	T1	2025
SAINT [102]	PAR-4	T5	2025
SWE-Gym [99]	PAR-7	T6	2025
Parasaram et al. [104]	PAR-2	T6	2025
SWE-MERA [105]	PAR-7	T6	2025
RepoHyper [106]	PAR-3	T1	2025
AgentFL [107]	PAR-7	T2	2025
Qin et al. [108]	PAR-1	T2	2025
Rahman&Rahman [109]	PAR-2	T1	2025
SpecRover [110]	PAR-2	T6	2025
Sedov et al. [111]	PAR-2	T1	2025
Shen et al. [112]	PAR-2	T1	2025
LongCodeZip [113]	PAR-5	T1	2025
Sohrabizadeh et al. [115]	PAR-7	T2,T6	2025
Tan et al. [116]	PAR-2	T1	2025

Continued on next page

Table 9 – continued

Citation	Primary Tasks PAR	Year
CGM [117]	PAR-3 T1,T3	2025
Tu et al. [120]	PAR-7 T5	2025
Vergopoulos et al. [121]	PAR-2 T3	2025
OpenHands [127]	PAR-6 T3,T6	2025
Wang et al. RAG [124]	PAR-2 T1	2025
RLCoder [129]	PAR-4 T1	2025
Wang et al. teach [123]	PAR-7 T3	2025
GRACE [131]	PAR-3 T1	2025
Divide-Conquer [128]	PAR-7 T6	2025
SWE-RL [132]	PAR-4 T6	2025
Demystify [135]	PAR-2 T6	2025
CoSrch [138]	PAR-1 T7	2025
ClassEval-T [139]	PAR-2 T4	2025
Deep Dive RAG [143]	PAR-2 T1	2025
Yang Empirical [142]	PAR-2 T1	2025
AskGraph [144]	PAR-3 T2	2025
OrcaLoca [145]	PAR-7 T2	2025
Yusuf et al. [146]	PAR-2 T1	2025
CodeS [147]	PAR-7 T3	2025
Multi-SWE-bench [148]	PAR-7 T6	2025
CITYWALK [159]	PAR-4 T5	2025
CodeRAG [160]	PAR-2 T1	2025
DI-BENCH [152]	PAR-3 T1	2025
HierCtxPrune [156]	PAR-5 T1	2025
PATCH [161]	PAR-7 T6	2025
Ref-RAG tests [157]	PAR-2 T5	2025
SWE-bench Live [153]	PAR-7 T6	2025
Scalable Trans. [150]	PAR-7 T4	2025
FastCoder [162]	PAR-2 T3	2025
HumanEvo [163]	PAR-2 T3	2025
LocAgent [19]	PAR-3 T2	2025
RepoScope [80]	PAR-3 T3	2025
CodexGraph [79]	PAR-3 T2,T3	2025
M2RC-EVAL [75]	PAR-2 T1	2025
ProjectEval [78]	PAR-7 T3	2025
Synergizing [2]	PAR-3 T2	2026
ArtifactSync [6]	PAR-3 T3	2026
SWE-QA-Pro [15]	PAR-7 T2	2026
Le-Anh et al. [59]	PAR-2 T3	2026
KTester [65]	PAR-4 T5	2026
RECODE-H [89]	PAR-6 T3	2026
Issue2Test [91]	PAR-7 T5	2026
Olewicki et al. [93]	PAR-2 T6	2026

Continued on next page

Table 9 – continued

Citation	Primary Tasks PAR		Year
RefAgent [96]	PAR-7	T3	2026
Prometheus [100]	PAR-7	T2,T6	2026
Tao RAG Survey [118]	PAR-2	T1	2026
Agent Ensemble [119]	PAR-7	T6	2026
Xiang et al. [136]	PAR-7	T6	2026
Hybrid-Gym [137]	PAR-7	T3,T6	2026
FeatureBench [164]	PAR-7	T3	2026
RPG [83]	PAR-3	T3	2026
RPG-Encoder [82]	PAR-3	T3	2026
C-to-Rust [32]	PAR-3	T4	2026
In-Line [49]	PAR-2	T3	2026
RepoTransBench [130]	PAR-2	T4	2026
SWE-Pruner [122]	PAR-5	T6	2026
CodeMEM [125]	PAR-6	T3	2026

B Quality Assessment Scores

Table 10 presents the aggregated quality assessment scores across the eight QA dimensions for the 160 primary studies. The first column gives the QA question identifier, the second the mean score across all studies, the third the standard deviation, and the fourth the fraction of studies that scored 1 (full credit) on that question.

Table 10. Aggregated quality assessment scores (N=160).

QA Dimension	Mean	SD	Score=1 (%)
QA1: RQ clarity	0.91	0.22	89.4
QA2: PAR description	0.82	0.30	78.8
QA3: Benchmark public	0.74	0.35	68.1
QA4: Metrics approp.	0.88	0.24	85.6
QA5: Baseline comp.	0.85	0.27	81.9
QA6: Threats validity	0.61	0.39	51.9
QA7: Implementation	0.58	0.41	48.1
QA8: Conclusions valid	0.87	0.26	83.8
Overall	6.26	1.14	–

The weakest dimensions are QA6 (Threats to validity, 62% of maximum) and QA7 (Implementation availability, 58% of maximum). The low score on QA7 is consistent with the finding noted in Section 5.5 that approximately 22% of systems do not release code artifacts. The low score on QA6 reflects a broader tendency in the LLM-based software engineering community to under-report validity threats—a pattern that has been observed in other systematic reviews of this field.

C Data Extraction Template

For completeness, Table 11 presents the full data extraction template used in this review, with the operational definition of each field and any controlled vocabularies used.

Table 11. Data extraction template with operational definitions.

Field	Operational definition	Notes / Controlled vocabulary
F1 Citation	Author, title, year, venue, DOI/URL	Standard BibTeX fields
F2 Primary PAR	Dominant representation paradigm	PAR-1 through PAR-7
F3 Secondary PAR(s)	Additional paradigms in hybrid	PAR-1 through PAR-7, comma-separated
F4 SE Task(s)	Repository-level tasks addressed	T1–T7, comma-separated
F5 LLM Backbone	Core generative model(s)	Model name and version
F6 Benchmarks	Evaluation datasets used	Dataset name(s)
F7 Primary Metric	Main reported evaluation metric and value	Metric name, value, e.g. EM=34.2
F8 Baselines	Comparison systems	Names of baselines
F9 Scalability Evidence	Reports on repos of varying size	Boolean
F10 Limitations	Authors acknowledge representation limitations	Boolean